

// JavaJub

Собес в Сбере

Стажировка SberSeasons

Java · Стажёр

Вопросы, задачи и подготовка
к интервью на студенческую стажировку

Стек

Java Core · ООП · Коллекции · SQL · Алгоритмы · Spring · Git

Содержание

01	О стажировке и формате собеседа	3
02	Стек: к чему готовиться	5
03	Java Core и ООП	6
04	Коллекции	11
05	JVM, память и GC	14
06	Многопоточность (базовое понимание)	16
07	БД и SQL	19
08	Spring и Spring Boot	23
09	Hibernate / JPA (минимум)	26
10	HTTP и REST	28
11	Git, Maven, Liquibase	30
12	Паттерны проектирования	32
13	Тесты — JUnit и Mockito	34
14	Алгоритмы и структуры данных	35
15	Практические задачи (live coding)	38
16	SQL live coding	44
17	Code Review (что любят давать в Сбере)	46
18	Pet-проекты для портфолио	48
19	Soft skills и рассказ о себе (STAR)	50
20	План подготовки и финальный чек-лист	52

01 О стажировке и формат собеса

SberSeasons — основная программа оплачиваемых стажировок Сбера для студентов очной формы (бакалавриат с 2 курса, специалитет, магистратура, аспирантура). Проходит два раза в год — весной и осенью. Стажировка длится 3–6 месяцев, а самое главное: 78% стажёров получают оффер в штат. То есть это не «отметка в резюме», а реальная дверь в IT-карьеру.

В этом гайде разобраны все вопросы и задачи, которые встречаются на интервью на стажировку по направлению Java. Уровень — стажёр / trainee (фактически Junior без коммерческого опыта). Сбер сам прямо подчёркивает: «не нужен опыт production-кода, нужны фундамент и желание учиться». Поэтому фокус — на основах Java Core, коллекциях, SQL, алгоритмах и умении думать вслух.

► Параметры программы

Параметр	Значение
Программа	SberSeasons — основная стажировка Сбера для студентов
Кому	Студенты очной формы: бакалавриат с 2 курса, специалитет, магистратура, аспирантура
Длительность	3–6 месяцев, обычно по 20+ часов в неделю
Зарплата	~45 000 ₽/мес (средняя стипендия, для аспирантов выше)
Формат	Офис / гибрид / удалёнка — зависит от команды и города
Оффер после	78% стажёров получают предложение в штат
Когда подают	Заявки два раза в год — весной и осенью на sberstudent.ru

► Как устроен отбор

Процесс полностью онлайн, 4–6 этапов. По времени растягивается на 2–4 недели.

Этап	Длит.	Что проверяют
1. Анкета на sberstudent.ru	1 раз	Резюме + мотивация + выбор направления. Здесь отсеивается больше всего кандидатов!
2. Онлайн-тесты	30–60 мин	Логика, английский, иногда базовый Java/SQL
3. Видео-интервью с HR	20–30 мин	Мотивация, опыт по STAR, базовый английский
4. Техническое интервью	30–90 мин	Java Core + алгоритмы (live coding) + SQL + Spring базово + soft
5. Доп. задание (опц.)	дни	Рет-задача: написать REST-сервис, тесты, выложить в Git
6. Финал / fit-интервью	30–45 мин	Рассказ о проекте + взаимная оценка с тимлидом

ВНИМАНИЕ · Анкета — главный отсев

Большинство кандидатов отсеивается на анкете. Это не формальность. Отнесись к ней как к резюме: распиши учебные проекты, курсовые, хакатоны, ссылки на GitHub, любой пет-проект. Если у тебя есть хоть один проект — обязательно укажи ссылку. Один работающий проект на GitHub стоит десяти строк в разделе «навыки».

► На что Сбер обращает особое внимание

1. Фундамент. ООП, коллекции, JVM «на пальцах», базовый SQL. Глубина промышленных знаний не нужна — нужна крепкая база.
2. Алгоритмы и структуры данных. Сбер сам подчёркивает: эта секция часто весомее, чем вопросы по Spring. К live coding надо тренироваться отдельно.
3. Способность думать вслух. На лайвкодинге интервьюер слушает ход мысли. Молчаливое верное решение хуже громкого неверного с правильными рассуждениями.
4. Готовность учиться. Вопросы намеренно задают за пределами твоих знаний, чтобы посмотреть, как ты реагируешь. Честное «не знаю, но логически предположил бы...» — лучше выдуманного ответа.
5. Адекватность. Не приукрашивай резюме. Опытные руководители на 5-минутной беседе видят, где правда, а где надувательство.
6. Командность. Расскажи про учебные проекты как про командную работу: «как договаривались», «как делили задачи», «как разрешали конфликты».

ФИШКА · Что говорить про Сбер на вопрос «почему именно мы»

Подготовь 2–3 факта: один из крупнейших работодателей IT в России, своя экосистема (СберМаркет, Самокат, СберЗдоровье, СберДевайсы), Platform V — собственная low-code платформа, активная разработка AI (GigaChat, Kandinsky). Можешь упомянуть конкретный продукт Сбера, которым пользуешься.

Главное — не говорить «потому что большая зарплата». Это правда, но это плохой ответ.

► Что Сбер НЕ ждёт от стажёра

Это важно знать, чтобы не нервничать. От стажёра не ждут:

- Глубокого знания System Design. Это не для стажировки.
- Production-опыта с Kafka, Kubernetes, микросервисами. Достаточно «знаю концепции, могу рассказать общими словами».
- Знания специфики банка (АБС, ЦБ РФ, проводки, межбанк) — научат на месте.
- Глубоких алгоритмов уровня LeetCode Hard. Задачи на стажировке — Easy с LeetCode.
- Идеального ответа на каждый вопрос. Честное «не знаю» — лучше попытки выкрутиться.

02 Стек: к чему готовиться

Сбер — большая компания с тысячами продуктовых команд. Стек разнится от команды к команде, но базовый набор, к которому проверяют стажёра, у всех одинаковый. Ниже — три группы: то, что точно спросят (must-have), то, что встречается часто, и то, что в плюс.

► Точно спросят (фундамент)

- Java Core: ООП, примитивы, String, исключения, Stream API, Optional, лямбды, функциональные интерфейсы.
- Коллекции: List / Set / Map, устройство HashMap, ArrayList vs LinkedList.
- JVM на пальцах: где живут примитивы и объекты, GC, разница JVM / JRE / JDK.
- Многопоточность базово: что такое поток, deadlock, synchronized, volatile.
- SQL: SELECT, JOIN, GROUP BY, HAVING, индексы, ACID, транзакции и уровни изоляции.
- Алгоритмы: сортировки, бинарный поиск, Big-O, стек, очередь.
- Git: базовый набор команд, что такое Pull Request.

► Часто встречается

- Spring и Spring Boot — на уровне «знаю, что такое DI и IoC, как работает @Autowired».
- Hibernate / JPA — спецификация vs реализация, @Entity, Lazy vs Eager, N+1 проблема.
- HTTP и REST — методы, идемпотентность, статус-коды, REST vs SOAP.
- Maven — pom.xml, жизненный цикл, scopes.
- Паттерны: Singleton, Builder, Factory, Observer, Proxy vs Decorator.
- Тесты: JUnit 5, Mockito, виды тестов (unit / integration).
- Liquibase / Flyway — что это и зачем (миграции БД).

► В плюс, если знаешь

- Kafka — основные понятия (topic, broker, producer, consumer). От стажёра не ждут глубины, но термины должны быть на слух.
- Docker — что такое контейнер, отличие от VM.
- Redis — что это (key-value хранилище).
- NoSQL базово — MongoDB (документная), Cassandra (column-family).

ФИШКА · Сбер использует Oracle И PostgreSQL

Это особенность Сбера — много legacy на Oracle, новые сервисы на PostgreSQL. Поэтому на себе SQL могут спросить через призму обеих БД: «в Oracle есть SEQUENCE, а в Postgres что?» (SERIAL / IDENTITY).

На стажёре глубокая разница не нужна, но 2–3 факта стоит знать: в Postgres нет Read Uncommitted, минимум — Read Committed; в Oracle есть BEFORE / AFTER триггеры; auto_increment в обеих БД делается по-разному.

► Что отличает Сбер

Платформа V — внутренняя облачная платформа Сбера, на которой работает значительная часть продуктов. Подразделение, в которое возьмут стажёра, скорее всего связано с одним из крупных направлений: Sber AI (GigaChat, Kandinsky), Платформа V (PaaS / IaaS), Экосистема (СберМаркет, СберЗдоровье, СберДевайсы), Корпоративные системы, Сопровождение IT-блока.

Знать всё это не нужно, но полезно зайти на developers.sber.ru и пробежать новости — это даст контекст и понимание, чем занимается компания.

ВНИМАНИЕ · Сбер не пользуется LeetCode Hard

По словам самих ребят из Сбера (статья на Хабре), на стажёре дают Easy-задачи с LeetCode. Это значит — сортировки, поиск, разворот строки, FizzBuzz, две суммы, палиндром. Не тратить время на Medium / Hard. Тратить на 30–50 Easy-задач, чтобы рука была набита.

03 Java Core и ООП

Самый большой блок на любом Junior-собесе. На стажировку в Сбер ждут уверенного владения базой: 4 принципа ООП, что такое JVM/JRE/JDK, как работают модификаторы доступа, в чём разница final / finally / finalize, как работает try-with-resources. Каждый ответ ниже — это минимум, который должен звучать на собеседовании.

▶ ООП — 4 принципа

- Назови 4 принципа ООП.

→ Инкапсуляция — скрывание внутреннего состояния объекта, доступ только через методы. Наследование — один класс получает поля и методы другого через extends. Полиморфизм — один интерфейс, разные реализации; вызов метода по ссылке на родителя выполняет нужный метод потомка. Абстракция — выделение существенного, скрывание деталей реализации (абстрактные классы и интерфейсы).

- Что такое DRY, KISS, YAGNI?

→ DRY — Don't Repeat Yourself. Не дублируй код, выноси в отдельный метод/класс. KISS — Keep It Simple, Stupid. Простое решение лучше навороченного. YAGNI — You Aren't Gonna Need It. Не пиши код «на будущее», который не нужен сейчас.

▶ SOLID

- Расшифруй SOLID.

→ S — Single Responsibility (один класс — одна ответственность). O — Open/Closed (открыт для расширения, закрыт для модификации — добавляй наследников, не правь оригинал). L — Liskov Substitution (наследника можно поставить вместо родителя без поломок поведения). I — Interface Segregation (много специализированных интерфейсов лучше одного большого). D — Dependency Inversion (зависим от абстракций, не от конкретных реализаций — это основа DI в Spring).

▶ JVM, JRE, JDK

- Чем отличаются JVM, JRE и JDK?

→ JVM (Java Virtual Machine) — виртуальная машина, которая исполняет байткод. Содержит GC и JIT-компилятор. JRE (Java Runtime Environment) — JVM + стандартные библиотеки (java.lang, java.util и т.д.), то что нужно для запуска. JDK (Java Development Kit) — JRE + компилятор javac + инструменты (jar, javadoc, jdb). JDK нужен для разработки, JRE — только для запуска.

▶ Прimitives и обёртки

- Какие примитивные типы есть в Java?

→ Их 8: byte (1 байт), short (2 байта), int (4 байта), long (8 байт), float (4 байта), double (8 байт), char (2 байта, UTF-16), boolean (1 бит). У каждого примитива есть класс-обёртка: Integer, Long, Boolean и так далее.

- Где хранятся примитивы и где объекты?

→ Примитивы как локальные переменные — на стеке. Если примитив — поле объекта, он лежит внутри объекта в куче. Все объекты — в куче (heap). Статические поля и метаданные классов — в Metaspace (с Java 8 заменил PermGen).

- Что такое автобоксинг и unboxing? В чём подвох?

→ Автоматическое преобразование примитива в обёртку и обратно: `Integer i = 5;` — это `int 5` заворачивается в `Integer` (boxing). `int j = i;` — обратно (unboxing). Подвох в том, что если обёртка `null`, при unboxing будем `NullPointerException`: `Integer i = null;` `int j = i;` — НПЕ.

ФИШКА · Integer cache

`Integer` кэширует значения от -128 до 127. Поэтому `Integer.valueOf(100) == Integer.valueOf(100)` даёт `true` (одна и та же ссылка из кэша). А `Integer.valueOf(200) == Integer.valueOf(200)` — `false` (разные объекты).

Правило: для обёрток ВСЕГДА используй `equals()`, не `==`. Это любимый подвох на собеседах.

► Object и его методы

- От какого класса наследуются все классы в Java?

→ От `java.lang.Object`. Это корень иерархии — даже массивы и `enum`'ы наследуются от него косвенно.

- Какие основные методы у `Object`?

→ `equals(Object)` — проверка равенства. `hashCode()` — целочисленный хэш. `toString()` — строковое представление. `getClass()` — `Class`-объект. `clone()` — копия объекта (нужно `implements Cloneable`). `wait()`, `notify()`, `notifyAll()` — для синхронизации потоков. `finalize()` — устаревший, deprecated с Java 9.

► equals и hashCode (классика)

- Расскажи контракт `equals` и `hashCode`.

→ Контракт `equals`: рефлексивность (`a.equals(a) = true`), симметричность (`a.equals(b) ⇔ b.equals(a)`), транзитивность (`a=b` и `b=c` → `a=c`), консистентность (повторный вызов даёт тот же результат), `equals(null) = false`. Главное правило связки: если `a.equals(b)`, то `a.hashCode() == b.hashCode()`. Обратное не обязательно выполняться.

- Что будет, если переопределить только `equals`, но не `hashCode`?

→ Сломаются `hash`-структуры: `HashMap`, `HashSet`, `Hashtable`. Положили объект — посчитался один `hashCode`, переопределённый `equals` говорит «они равны», но другой `hashCode` значит другой бакет. Не найдёшь, что положил. Поэтому при переопределении `equals` — обязательно переопределяй `hashCode`.

- Что будет, если `hashCode` возвращает константу (например, `return 1`)?

→ Все объекты попадут в один бакет `HashMap`. Поиск превратится в перебор связанного списка (или дерева с Java 8) — деградация с $O(1)$ до $O(n)$ или $O(\log n)$. Карта работает, но медленно. Это любимый подвох на собеседе.

► String и его особенности

- Почему `String` immutable?

→ Безопасность: пароли, URL, `classpath` — нельзя случайно изменить. `String Pool` — иммутабельность нужна для безопасного шеринга. Хэш-структуры: `hashCode` у `String` кэшируется, нельзя ломать. Потокбезопасность: иммутабельный объект можно безопасно шарить между потоками без синхронизации.

- `==` vs `equals` для строк?

→ `==` сравнивает ссылки. `equals` — содержимое. Из-за `String Pool` два литерала `"abc" == "abc"` дадут `true`, но `new String("abc") == "abc"` — `false` (`new` создаёт новый объект вне пула). На собеседе всегда отвечать «`equals` для содержимого».

- В чём отличие `String`, `StringBuilder`, `StringBuffer`?

→ *String* — *immutable*. Каждая конкатенация создаёт новый объект, в цикле это $O(n^2)$ по памяти. *StringBuilder* — *mutable*, не потокобезопасный, быстрый. *StringBuffer* — *mutable* + *synchronized* методы, потокобезопасный, но медленнее. В одном потоке всегда нужен *StringBuilder*.

► Модификаторы доступа

- Какие модификаторы доступа есть в Java?

→ *private* — только внутри своего класса. *package-private* (без модификатора, *default*) — внутри своего пакета. *protected* — пакет + наследники из других пакетов. *public* — отовсюду.

- Как работает *protected* через пакеты?

→ *protected*-метод виден всему своему пакету (как *package-private*) И всем наследникам, даже из других пакетов. То есть из другого пакета можно получить доступ только через наследование, не напрямую через ссылку. Тонкость на себе.

► **static, final**

- Что такое *static*?

→ Принадлежит классу, а не объекту. *static*-поле общее для всех экземпляров. *static*-метод можно вызвать без объекта: *ClassName.method()*. *static*-блок выполняется при загрузке класса. Часто используется для констант (*static final*), фабричных методов, утилит.

- Что делает *final* для класса, метода, переменной?

→ Для класса — нельзя унаследовать (*final class String*). Для метода — нельзя переопределить в наследнике. Для переменной — можно присвоить один раз. Для поля — становится константой (часто вместе со *static*).

► **final, finally, finalize — три разных слова**

- В чём разница *final*, *finally*, *finalize*?

→ *final* — модификатор. *final class / method / field*. *finally* — блок в *try-catch-finally*, выполняется ВСЕГДА (даже при исключении или *return*). Используется для освобождения ресурсов. *finalize()* — метод *Object*, который раньше вызывал GC перед удалением объекта. *Deprecated* с Java 9, использовать нельзя. Сейчас для очистки используют *try-with-resources* и *Cleaner*.

► **abstract class vs interface**

- В чём разница абстрактного класса и интерфейса?

→ Абстрактный класс может иметь состояние (поля), конструктор, реализованные методы. От него можно унаследоваться ТОЛЬКО одного. Интерфейс — контракт, описывающий «что умеет». С Java 8 могут быть *default*-методы (с реализацией) и *static*-методы. Можно реализовать ЛЮБОЕ количество интерфейсов. Использовать абстрактный класс — когда есть общая база и состояние. Интерфейс — когда нужно описать роль или *behaviour*.

- Что такое *default*-методы в интерфейсах?

→ С Java 8 в интерфейсе можно объявить метод с реализацией через ключевое слово *default*. Это решает проблему расширения интерфейсов без поломки совместимости: добавляешь новый метод с *default*-реализацией, и существующие реализации не ломаются. Так в Java 8 в *Collection* появился *stream()* — *default*-метод.

► **Исключения**

- Расскажи иерархию исключений.

→ *Throwable* — корень. Делится на *Error* (*OOM*, *StackOverflow* — не ловить) и *Exception*. *Exception* → *checked* (*IOException*, *SQLException* — обязаны быть в *throws*) и *RuntimeException* → *unchecked* (*NullPointerException*, *IllegalArgumentException* — не обязаны).

- Что такое checked и unchecked исключения?

→ *Checked* — компилятор требует обработать (catch) или объявить в throws. Используются для ожидаемых внешних ошибок (IO, БД). *Unchecked* (RuntimeException и наследники) — обрабатывать не обязательно. Используются для ошибок программирования (NPE, IllegalArgumentException).

- Можно ли создать свой exception от RuntimeException?

→ *Да, и это типовая практика. Большинство современных проектов делают свои исключения от RuntimeException — не загромождают сигнатуру методов throws. От Error не рекомендуется наследоваться — это «системные» ошибки JVM, бизнес-код их трогать не должен.*

- Что такое try-with-resources?

→ *Конструкция, которая гарантирует вызов close() у ресурсов, реализующих AutoCloseable. Заменяет try-finally. При исключении в try ресурсы всё равно закроются, а если close() сам бросит — оно станет suppressed внутри основного.*

```
try (BufferedReader reader = new BufferedReader(new FileReader("file.txt"))) {  
    return reader.readLine();  
} // reader.close() вызовется автоматически  
// Даже если упадёт readLine — close всё равно сработает
```

► Optional

- Для чего нужен Optional?

→ *Контейнер, который может содержать или не содержать значение. Появился в Java 8 как альтернатива null. Помогает явно показывать в сигнатуре методов, что результат может отсутствовать, и заставляет вызывающего обработать оба случая. Цель — уменьшить NullPointerException.*

- В чём разница orElse и orElseGet?

→ *orElse(value) — eager: значение вычисляется ВСЕГДА, даже если Optional не пустой. orElseGet(Supplier) — lazy: вызывается только если Optional пустой. Если default-значение получается дорогим вызовом — использовать orElseGet.*

```
Optional<User> user = repo.findById(id);  
  
// ❌ Плохо: heavyDefault() вызывается всегда  
User u1 = user.orElse(heavyDefault());  
  
// ✅ Хорошо: heavyDefault() вызывается только при пустом Optional  
User u2 = user.orElseGet(() -> heavyDefault());
```

► Лямбды и функциональные интерфейсы

- Что такое функциональный интерфейс?

→ *Интерфейс с РОВНО ОДНИМ абстрактным методом (default и static не считаются). Можно реализовать лямбдой. @FunctionalInterface — необязательная аннотация для compile-time проверки.*

- Какие функциональные интерфейсы из java.util.function ты знаешь?

→ *Consumer<T> — принимает T, ничего не возвращает (T → void). Supplier<T> — без параметров, возвращает T (() → T). Predicate<T> — принимает T, возвращает boolean. Function<T,R> — T → R. BiFunction<T,U,R> — два параметра. UnaryOperator<T> = Function<T,T>. BinaryOperator<T> = BiFunction<T,T,T>.*

- Какие переменные можно захватывать в лямбде?

→ *Только final или effectively final (не меняются после присваивания). Поля класса можно использовать без ограничений. Локальные переменные — только если они эффективно финальные. Это нужно для гарантии корректности при асинхронном вызове.*

► Stream API

- Что такое Stream API?
 - Введён в Java 8. Конвейерная обработка коллекций в декларативном стиле. Поток (Stream) — это последовательность элементов с операциями. Не путать с потоком (Thread) — это про многопоточность.
- Какие операции бывают в Stream?
 - Промежуточные (intermediate) — возвращают Stream, lazy: filter, map, flatMap, distinct, sorted, peek. Терминальные (terminal) — запускают конвейер: collect, reduce, forEach, count, findFirst. Без терминальной операции Stream НЕ выполняется.
- Что значит lazy в Stream?
 - Промежуточные операции не выполняются сразу. Они «копят» план. Терминальная операция запускает выполнение. Поэтому stream.filter(...).map(...) без terminal — не сделает ничего. Это позволяет оптимизировать выполнение.

► Inkrement

- Что делает i++?
 - Read-modify-write: прочитать значение, добавить 1, записать обратно. Это ТРИ операции, и они НЕ атомарны. Поэтому в многопоточной среде два потока могут одновременно прочитать одно значение и оба прибавить — итог будет +1 вместо +2. Для атомарности нужно AtomicInteger или synchronized.

► Records, var (новое в Java)

- Что такое record (Java 14+)?
 - Короткий способ описать иммутабельный DTO. Автоматически генерируются: конструктор, геттеры (имя поля без get-), equals, hashCode, toString. record User(String name, int age) {} — всё. Меньше boilerplate, чем Lombok.
- Что такое var (Java 10+)?
 - Ключевое слово для вывода типа локальной переменной. var list = new ArrayList<String>(); — компилятор сам определит, что это ArrayList<String>. Использовать только для локальных переменных, не для полей и параметров. И только когда тип очевиден из контекста (var x = 5 — плохо, var users = repo.findAll() — норм).

► System.exit и потоки I/O

- Что делает System.exit()?
 - Принудительно завершает JVM с кодом возврата. Не выполняются finally-блоки, не закрываются ресурсы. Использовать ТОЛЬКО в экстренных случаях. В обычной разработке — не нужен.
- Как работает чтение файлов в Java?
 - Через потоки I/O: InputStream / OutputStream — для байтов. Reader / Writer — для символов (с учётом кодировки). Buffered-обёртки (BufferedReader, BufferedInputStream) — для эффективности, читают сразу пачку, потом отдают по запросу. Использовать обязательно в try-with-resources, чтобы закрыть.

04 Коллекции

Обязательный блок на любом Junior-собесе. На Сбере особенно любят вопросы про HashMap (что внутри, что при коллизии), разницу ArrayList и LinkedList, что такое ConcurrentModificationException.

► Иерархия

- Расскажи иерархию коллекций.
 - *Iterable* → *Collection*. От *Collection* наследуются *List*, *Set*, *Queue*. *Map* стоит ОТДЕЛЬНО — она НЕ *Collection*. Это пары ключ-значение, не отдельные элементы. Поэтому у *Map* свои методы (*put*, *get*), и она не реализует *Collection*.
- Почему *Map* отдельно от *Collection*?
 - *Collection* хранит элементы — единичные значения. *Map* хранит ПАРЫ ключ-значение, у неё другой API: *put(key, value)*, *get(key)*, *entrySet()*. Хотя по сути обе — структуры данных.

► List и его реализации

- Какие реализации *List* ты знаешь?
 - *ArrayList* — на основе массива. Доступ по индексу $O(1)$, вставка в конец амортизированно $O(1)$, в середину $O(n)$. Самая популярная. *LinkedList* — двусвязный список. Доступ по индексу $O(n)$, вставка/удаление в любом месте — $O(1)$ при наличии итератора. *Vector* — *synchronized* аналог *ArrayList*, legacy, не используется.
- Чем отличается *List* от *ArrayList*?
 - *List* — интерфейс, *ArrayList* — реализация. Правило: писать переменные через интерфейс — *List<String> list = new ArrayList<>()*; — чтобы потом можно было сменить реализацию без правок остального кода.
- Сравни *ArrayList* и *LinkedList* по сложности.
 - Доступ по индексу: *ArrayList* $O(1)$, *LinkedList* $O(n)$ (нужно идти от головы или хвоста). Вставка в конец: *ArrayList* амортизированный $O(1)$, *LinkedList* $O(1)$. Вставка в середину: *ArrayList* $O(n)$ (сдвиг элементов), *LinkedList* $O(n)$ на поиск + $O(1)$ на саму вставку.
- Почему *ArrayList* на практике быстрее *LinkedList*?
 - *Cache locality*. *ArrayList* хранит элементы в непрерывном куске памяти — процессор подгружает целые блоки в L1/L2 кэш. У *LinkedList* узлы разбросаны по куче — каждый *next()* это *cache miss*. На реальных данных *ArrayList* выигрывает почти всегда. *LinkedList* в современном коде использовать не надо — для очередей есть *ArrayDeque*.
- Как растёт *ArrayList*?
 - Начальная *capacity* = 10. При переполнении создаётся новый массив размером 1.5× от старого, элементы копируются. Метод *trimToSize()* ужимает массив до текущего количества элементов. Если заранее знаешь приблизительный размер — задавай в конструкторе: *new ArrayList<>(expectedSize)*.

► Set и его реализации

- Какие реализации *Set*?
 - *HashSet* — на основе *HashMap* (значение в виде специальной заглушки). Не сохраняет порядок. *LinkedHashSet* — *HashSet* + порядок вставки. *TreeSet* — на основе *TreeMap* (красно-чёрное дерево), элементы отсортированы. Все три не потокобезопасны.
- Чем *HashSet* отличается от *LinkedHashSet*?

→ *HashSet* — без гарантии порядка. *LinkedHashSet* хранит порядок добавления. Дополнительная память на двусвязный список, но итерация предсказуемая. Часто используют как «уникальные значения с сохранением порядка».

► **Мар и его реализации**

- Расскажи про реализации Мар.

→ *HashMap* — основная, не потокобезопасна, без порядка, *null*-ключ можно. *LinkedHashMap* — порядок вставки или *access-order* (последнее нужно для LRU-кэша). *TreeMap* — отсортирована по ключу, через *NavigableMap* (*firstKey*, *lastKey*, *floorKey*). *Hashtable* — *legacy*, *synchronized*, *null*-ключи не разрешены, не используется.

► **HashMap (самый частый вопрос)**

- Как устроена HashMap внутри?

→ Массив бакетов (*table*). По хэшу ключа определяется индекс бакета (примерно: $\text{hash} \bmod \text{capacity}$). В бакете — связный список узлов *Node* ($\text{hash} + \text{key} + \text{value} + \text{next}$). С Java 8: если в одном бакете ≥ 8 элементов И размер таблицы ≥ 64 — связный список превращается в красно-чёрное дерево (*treeify*). Это защита от плохих хэшей. Когда бакет уменьшается до 6 — дерево разворачивается обратно в список.

- Как работает put / get в HashMap?

→ *put*: вычислили *hashCode* ключа → определили бакет → в бакете ищем по *equals* существующий ключ. Если нашли — обновляем значение. Если нет — добавляем новый *Node*. После добавления проверяем условия для *resize* ($\text{size} > \text{capacity} * 0.75$) и для *treeify*. *get*: *hashCode* → бакет → перебор по *equals* → значение.

- Что такое load factor?

→ Соотношение $\text{size} / \text{capacity}$, при котором происходит расширение таблицы. По умолчанию 0.75. То есть когда *size* превышает 75% от *capacity*, таблица расширяется в 2 раза и все элементы перехэшируются. Уменьшение *load factor* — меньше коллизий, больше памяти. Увеличение — наоборот.

- Какая сложность операций в HashMap?

→ Среднее: *get* / *put* / *containsKey* — $O(1)$. Худший случай при плохих хэшах: $O(\log n)$ при *treeified* бакете, $O(n)$ если *treeify* ещё не сработал.

- Можно ли null в HashMap?

→ *HashMap* — да, ОДИН *null*-ключ (лежит в *table[0]*), *null*-значений сколько угодно. *TreeMap* — *null*-ключ нельзя (бросит *NPE* при сравнении). *Hashtable* — *null* нельзя ни в ключе, ни в значении (*NPE*).

- Зачем переопределять hashCode и equals для ключа HashMap?

→ *HashMap* использует *hashCode* для бакета и *equals* для разрешения коллизий. Если по умолчанию (*Object*) — каждый объект уникален, два «равных по смыслу» *User*'а с одинаковым *id* будут в разных бакетах. Чтобы карта работала «по содержимому» — нужно переопределить оба метода согласованно.

ВНИМАНИЕ • Ключи HashMap должны быть иммутабельными

Если изменить поле, влияющее на *hashCode*, ПОСЛЕ того как положил объект в карту — объект «потеряется». При поиске *hashCode* даст другой бакет, объект не найдём.

Используй для ключей: *String*, *Integer*, *UUID*, или собственные *final*-классы без сеттеров.

► **ConcurrentModificationException**

- Что такое ConcurrentModificationException?

→ Бросается итератором, если коллекция изменилась во время итерации не через сам итератор. Реализуется через счётчик `modCount` внутри коллекции и `expectedModCount` внутри итератора. Если они разъезжаются — исключение.

```
// ❌ Так нельзя
List<String> list = new ArrayList<>(List.of("a", "b", "c"));
for (String s : list) {
    if (s.equals("b")) list.remove(s); // ConcurrentModificationException
}

// ✅ Через Iterator
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    if (it.next().equals("b")) it.remove();
}

// ✅ Через removeIf
list.removeIf(s -> s.equals("b"));
```

► Iterator и ListIterator

- Чем Iterator отличается от ListIterator?

→ *Iterator* — однонаправленный обход. Методы `hasNext`, `next`, `remove`. Доступен для любой *Collection*. *ListIterator* — только у *List*. Двухнаправленный (`hasPrevious` / `previous`), может добавлять (`add`) и заменять (`set`) элементы во время итерации.

► Comparable и Comparator

- В чём разница Comparable и Comparator?

→ *Comparable<T>* — внутри класса (метод `compareTo`). Описывает «естественный» порядок. Один на класс. *Comparator<T>* — отдельно. Можно сделать несколько разных *Comparator*'ов для одного класса. Если нужно сортировать *User* по имени, по возрасту, по *email* — это три разных *Comparator*'а, а не три `compareTo`.

- Как сортировать List по убыванию по полю?

→ `list.sort(Comparator.comparing(User::getAge).reversed())` или с лямбдой `list.sort((a, b) -> b.getAge() - a.getAge())` — но безопаснее через `Comparator.comparing`.

► Потокобезопасные коллекции

- Какие потокобезопасные коллекции есть в `java.util.concurrent`?

→ *ConcurrentHashMap* — потокобезопасная *HashMap*. Не блокирует чтение, на запись — синхронизация по бакету. *CopyOnWriteArrayList* — копирует весь массив при записи, *lock-free* на чтение. *BlockingQueue* — для *producer-consumer* (`put/take` блокируют, если очередь полна/пуста).

- Как сделать иммутабельный список?

→ `List.of(...)` — с Java 9, неизменяемый. `Collections.unmodifiableList(list)` — обёртка, но если изменить оригинальный `list` — обёртка тоже увидит изменения. `List.copyOf(list)` — полноценная иммутабельная копия.

05 JVM, память и GC

На стажёре глубокое знание JVM не ждут, но нужно уверенно отвечать «где живут примитивы и где объекты», «что такое heap и stack», «как примерно работает GC». Это типовый блок в любом Junior-собесе.

► Структура памяти Java

- Из чего состоит память Java?

- Основные области: *Heap* (куча) — здесь живут все объекты. *Stack* (стек) — у каждого потока свой, хранит локальные переменные и фреймы вызовов. *Metaspace* (с Java 8 заменил *PermGen*) — метаданные классов, статические поля. Также есть *Code Cache* (для JIT-кода) и *Direct Memory* (off-heap).

- Как делится Heap?

- *Young Generation* — новые объекты. Состоит из *Eden* (где создаются), *Survivor 0*, *Survivor 1* (для тех, что пережили несколько Minor GC). *Old Generation* — долгоживущие объекты, которые пережили много циклов GC.

- Где хранятся примитивы и где объекты?

- Примитив как локальная переменная — на стеке. Примитив как поле объекта — внутри объекта в куче. Все объекты — в куче. Ссылки на объекты — на стеке (как локальные) или в полях другого объекта (в куче).

- Где хранятся статические поля?

- В *Metaspace*. До Java 8 это была область *PermGen* (часть heap), с Java 8 — отдельная область в нативной памяти (off-heap). Это важно: при анализе утечек статические поля не видны в обычном дампе heap.

► StackOverflowError и OutOfMemoryError

- В чём разница StackOverflowError и OutOfMemoryError?

- *StackOverflowError* — переполнение стека. Возникает при бесконечной рекурсии или очень глубоких вызовах. *Стек* у потока ограничен (по умолчанию около 512 КБ — 1 МБ). *OutOfMemoryError* — нехватка памяти в куче (heap), *Metaspace*, *Direct Memory* или *Stack*. Чаще всего — heap. Возникает при утечке памяти, слишком больших объектах, слишком маленьком *Хтх*.

► Garbage Collector

- Как GC решает, что объект можно удалить?

- По достижимости от GC Roots. GC Roots — статические поля, локальные переменные стека, активные потоки, JNI-ссылки. Если до объекта нельзя добраться по ссылкам от GC Roots — он мусор и будет собран.

- Циклические ссылки — соберутся?

- Да. Java GC работает по reachability, а не по reference counting. Два объекта, ссылающиеся друг на друга, но не достижимые снаружи — будут собраны. Это отличие от Python, где есть reference counting + cycle collector.

- Какие сборщики мусора ты знаешь?

- *Serial* — однопоточный, для маленьких приложений. *Parallel* — многопоточный для Young, был дефолтом до Java 9. *G1* (Garbage First) — с Java 9 дефолт, делит heap на регионы, низкие паузы. *ZGC* и *Shenandoah* — для больших heap, паузы менее 10 мс. *Generational ZGC* — с Java 21, добавил поколения в ZGC.

- Что такое Minor GC и Major GC?

→ *Minor GC — собирает Young Generation. Быстро (миллисекунды), Stop-The-World, но коротко. Major GC — собирает Old Generation, медленнее. Full GC — весь heap + Metaspace. Тревожное событие, если случается часто.*

► **Виды ссылок**

- Какие виды ссылок в Java?

→ *Strong (обычная new) — пока есть Strong-ссылка, объект не соберётся. Soft (SoftReference) — собирается, когда не хватает памяти. Для кэшей. Weak (WeakReference) — собирается при следующей сборке, даже если есть память. Например, ключи WeakHashMap. Phantom (PhantomReference) — для post-finalization очистки, используется редко.*

06 Многопоточность (базовое понимание)

На стажёре спрашивают не глубоко: что такое поток, как создать, что такое deadlock, race condition, монитор, чем отличается volatile от Atomic. Глубже только если кандидат сам полезет в дебри.

► Поток vs процесс

- В чём разница процесса и потока?

→ Процесс — изолированный экземпляр программы. У каждого процесса своё адресное пространство, файловые дескрипторы, переменные окружения. Процессы не могут просто так читать память друг друга. Поток — единица выполнения внутри процесса. Все потоки одного процесса делят общую память и ресурсы. Поэтому потоки лёгкие, но требуют синхронизации.

► Как создать поток

- Какие способы создать поток в Java?

→ (1) Унаследоваться от Thread и переопределить run(). Не лучший способ — лимит одного наследования. (2) Реализовать Runnable и передать в Thread: `new Thread(() -> {...}).start();` (3) Реализовать Callable и отдать в ExecutorService — единственный способ вернуть результат и пробросить исключение. (4) ExecutorService с пулом — продакшен-вариант, не создавать потоки руками.

- В чём разница start() и run() у Thread?

→ start() — создаёт новый поток ОС и в нём вызывает run(). run() напрямую — это обычный метод, выполнится в ТЕКУЩЕМ потоке. Это любимая ловушка на собеседах. Запомни: «start запускает новый, run просто исполняет в этом же».

- Чем Runnable отличается от Callable?

→ Runnable — run() возвращает void, не может бросать checked-исключения. Подходит для «просто запусти». Callable<V> — call() возвращает V, может бросать Exception. Используется через ExecutorService.submit() и Future<V>.

► synchronized и монитор

- Что такое монитор?

→ У каждого Java-объекта есть неявный монитор — встроенный механизм блокировки. Только один поток в один момент может «владеть» монитором объекта. Это используется в synchronized.

- Как работает synchronized на методе?

→ Для обычного метода блокировка идёт на this (на сам объект). Для static-метода — на Class-объект (ClassName.class). На уровне байткода — это инструкции monitorenter / monitorexit. Если уже владеешь монитором, можешь повторно зайти (реентерабельный).

- Чем отличается synchronized на методе и в блоке?

→ synchronized на методе — блокировка на весь метод. synchronized(obj) {...} — на конкретный объект, на конкретный кусок кода. Блок даёт больше контроля: блокировку можно держать только на нужный участок, можно синхронизироваться на разных объектах.

► volatile

- Что гарантирует volatile?

→ Видимость записи между потоками — запись одного потока становится сразу видна другим. Запрет переупорядочивания инструкций вокруг `volatile`. Но `volatile` НЕ даёт атомарности составных операций. `i++` — это `read-modify-write`, не атомарно даже с `volatile`.

- Почему `i++` не атомарен?

→ Это три операции: прочитать значение → прибавить 1 → записать обратно. Два потока могут одновременно прочитать одно значение, оба прибавить, оба записать — будет +1 вместо +2 (*lost update*). Для атомарности — `AtomicInteger` или `synchronized`.

- ▶ **Atomic**

- Чем Atomic лучше volatile?

→ `AtomicInteger` / `AtomicLong` / `AtomicReference` дают и видимость, и атомарность через CAS (*compare-and-swap*). CAS — операция процессора «прочитать, проверить, заменить» одной инструкцией. `compareAndSet`, `incrementAndGet` — атомарны. Это *lock-free*.

- ▶ **wait / notify / notifyAll**

- Где можно вызывать `wait` и `notify`?

→ Только внутри `synchronized`-блока на том же объекте. `wait()` отпускает монитор, поток засыпает. `notify()` / `notifyAll()` — будит. После `notify` поток должен снова получить монитор, чтобы продолжить. Без `synchronized` — `IllegalMonitorStateException`.

- Что отпускает `wait`, а что — `sleep`?

→ `wait()` отпускает монитор объекта. `sleep()` — НЕ отпускает, поток продолжает держать все свои блокировки, просто не выполняется.

- ▶ **Race condition и deadlock**

- Что такое race condition?

→ Гонка — ситуация, когда результат зависит от порядка выполнения потоков. Классика: `i++` из двух потоков. Может быть +1, может быть +2 — зависит от планировщика. Решение — синхронизация (`synchronized`, `locks`, `atomic`) или `immutable`.

- Что такое deadlock?

→ Взаимная блокировка двух (или более) потоков. Поток А держит ресурс X, ждёт Y. Поток B держит Y, ждёт X. Никто не сдвинется.

- 4 условия возникновения deadlock?

→ (1) *Mutual exclusion* — ресурс не разделяемый. (2) *Hold and wait* — поток держит один ресурс, ждёт другой. (3) *No preemption* — отнять ресурс нельзя. (4) *Circular wait* — цикл ожидания. Чтобы deadlock был — нужны все четыре. Уберёшь одно — не будет deadlock.

- Как избежать deadlock?

→ Самое простое — захватывать ресурсы в едином порядке (например, по ID). Тогда циклический wait невозможен. Альтернатива — `tryLock` с таймаутом (`ReentrantLock`).

- ▶ **ThreadLocal**

- Для чего нужен ThreadLocal?

→ Хранение значения, уникального для каждого потока. Например, `traceId`, MDC для логов, контекст транзакции. В рамках одного потока — данные доступны без передачи через параметры. В пулах потоков обязательно вызывать `remove()` в `finally`, иначе значение «прилипнет» к потоку и достанется следующему пользователю.

- ▶ **ExecutorService и пулы**

- Зачем нужны пулы потоков?

→ Создание потока — дорогая операция (~1 МБ памяти + системный вызов). Пул переиспользует потоки. Также даёт контроль над количеством (не создаст тысячу) и очередь задач.

- Какие типы пулов есть в Executors?

→ `newFixedThreadPool(n)` — фиксированный размер. `newCachedThreadPool()` — растёт по необходимости, потоки умирают через 60 сек простоя. `newSingleThreadExecutor()` — один поток. `newScheduledThreadPool()` — для отложенных и периодических задач.

- ▶ **Проблема с нестатическими полями в сервлете**

- Если в обработчике (сервлете) сделать нестатическое поле — какие проблемы возникнут?

→ Сервлет один на JVM (по умолчанию), обрабатывает много запросов параллельно. Нестатическое поле — общее для всех запросов. Получится *shared mutable state* без синхронизации = *race condition*, гонка данных. Решение: (1) сделать поле локальным внутри метода (стек = безопасно). (2) Использовать `synchronized` / `volatile` / `Atomic`. (3) Использовать `ThreadLocal`. То же касается `@Service` / `@Component` бинов в Spring — они тоже *singleton*.

07 БД и SQL

На стажировке SQL спрашивают активно. Сбер использует и Oracle (legacy), и PostgreSQL (новые сервисы). База: SELECT, JOIN, GROUP BY, HAVING, индексы, ACID, транзакции. В разделе 16 — практические SQL-задачи, типичные для собесед.

► DML, DDL, DCL, TCL

- Какие группы команд SQL ты знаешь?

→ DML (Data Manipulation Language) — работа с данными: SELECT, INSERT, UPDATE, DELETE. DDL (Data Definition Language) — структура: CREATE, ALTER, DROP. DCL (Data Control Language) — права: GRANT, REVOKE. TCL (Transaction Control Language) — управление транзакциями: COMMIT, ROLLBACK, SAVEPOINT.

► Порядок выполнения SELECT

- В каком порядке выполняются части SELECT-запроса?

→ FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT. Это важно: WHERE применяется ДО агрегации, HAVING — после. Алиасы из SELECT можно использовать в ORDER BY, но не в WHERE.

► JOIN-ы

- Какие виды JOIN ты знаешь?

→ INNER JOIN — только совпадения из обеих таблиц. LEFT JOIN — все строки из левой + совпадения из правой (несовпадения = NULL). RIGHT JOIN — наоборот. FULL OUTER JOIN — все строки из обеих, несовпадения = NULL. CROSS JOIN — декартово произведение.

- LEFT JOIN: ON или WHERE — есть разница?

→ Да! Условие в ON применяется ДО объединения — несоответствующие строки правой таблицы становятся NULL. Условие в WHERE применяется ПОСЛЕ — строки с NULL отфильтровываются, и LEFT JOIN превращается в INNER JOIN. Пример: чтобы получить пользователей без заказов: LEFT JOIN orders ON o.user_id = u.id WHERE o.id IS NULL. WHERE o.id IS NULL после LEFT JOIN — правильно. Поместить в ON — даст всех пользователей и пустые поля заказов.

► WHERE и HAVING

- В чём разница WHERE и HAVING?

→ WHERE фильтрует строки ДО группировки. HAVING — после, по агрегатам. SELECT user_id, COUNT(*) FROM orders WHERE status='OK' GROUP BY user_id HAVING COUNT(*) > 5. По агрегатам (COUNT, SUM) фильтровать в WHERE НЕЛЬЗЯ — они ещё не вычислены.

► Агрегатные функции

- COUNT(*) — все строки. COUNT(col) — все строки, где col не NULL. COUNT(DISTINCT col) — уникальные значения.
- SUM, AVG, MIN, MAX — стандартные. NULL игнорируется.
- STRING_AGG (Postgres) / LISTAGG (Oracle) — конкатенация значений.

► DISTINCT, UNION

- В чём разница UNION и UNION ALL?

→ UNION — объединяет результаты ДВУХ SELECT, убирает дубликаты. UNION ALL — то же, но БЕЗ удаления дубликатов. UNION ALL быстрее (не нужно сортировать для дедупликации). Если знаешь, что дублей не будет — всегда UNION ALL.

► NULL в SQL

- Почему NULL = NULL даёт UNKNOWN, а не TRUE?

→ NULL — это «значение неизвестно». Два неизвестных значения — нельзя сказать, равны они или нет. Поэтому NULL = NULL это не TRUE, а UNKNOWN, в условиях ведёт себя как FALSE. Для проверки используется IS NULL.

► Ключи и индексы

- Что такое PRIMARY KEY и FOREIGN KEY?

→ PRIMARY KEY — уникальный идентификатор строки. NOT NULL, UNIQUE автоматически. На таблицу может быть только один. FOREIGN KEY — ссылка на PRIMARY KEY (или UNIQUE) другой таблицы. Поддерживает целостность данных: нельзя добавить заказ с user_id, которого нет в таблице users.

- В чём разница PRIMARY KEY и UNIQUE?

→ PRIMARY KEY — один на таблицу, NOT NULL. UNIQUE — может быть несколько UNIQUE-индексов, разрешает NULL (NULL не равно NULL, поэтому много NULL допустимо).

- Можно ли изменить первичный ключ?

→ Технически — да, через ALTER. Но фактически — нельзя, потому что: (1) на нём FK из других таблиц. (2) Индексы строятся по PK. (3) В кластерных индексах данные физически отсортированы по PK. Операция дорогая и опасная. На собесе правильный ответ: «по сути нет».

- Что такое SEQUENCE?

→ Oracle: SEQUENCE — отдельный объект БД, генератор уникальных чисел. Используется для auto-increment ID: INSERT VALUES (my_seq.nextval, ...). Postgres: аналог — SERIAL или IDENTITY. SERIAL — алиас для INTEGER + sequence. IDENTITY (SQL стандарт) — более явный. MySQL: AUTO_INCREMENT.

► Индексы

- Что такое индекс? Зачем он нужен?

→ Структура данных (обычно B-tree), позволяющая БД быстро находить строки по значениям колонок. Без индекса — Seq Scan (полный обход таблицы). С индексом — поиск за $O(\log n)$.

- Какие виды индексов есть?

→ B-tree (по умолчанию, для =, <, >, BETWEEN, LIKE 'abc%'). Hash (только =, в Postgres есть, но используется редко). GIN — для массивов, JSONB, full-text. GiST — геометрия, диапазоны. BRIN — для огромных таблиц с естественной упорядоченностью (временные ряды).

- Почему нельзя на все поля навесить индексы?

→ (1) Каждый индекс нужно обновлять при INSERT/UPDATE/DELETE — замедление записи. (2) Индексы занимают место — на больших таблицах могут весить больше самой таблицы. (3) Они требуют обслуживания (VACUUM, REINDEX). Правило: индексы только на колонки, по которым реально часто фильтруют, сортируют, джойнят.

► ACID и транзакции

- Что такое транзакция?

→ Последовательность операций, которая выполняется как единое целое. Либо все операции применяются (COMMIT), либо ни одна (ROLLBACK). Гарантии — ACID.

- Расскажи ACID.
 - *Atomicity* — атомарность: транзакция выполняется целиком или откатывается. *Consistency* — консистентность: БД переходит из одного валидного состояния в другое (констрейнты соблюдаются). *Isolation* — изоляция: параллельные транзакции не мешают друг другу. *Durability* — стойкость: после COMMIT данные сохраняются даже при сбое.

► Уровни изоляции

- Какие уровни изоляции бывают?
 - *Read Uncommitted* (можно читать незакоммиченные — *dirty read*). *Read Committed* (только закоммиченные, но возможен *non-repeatable read*). *Repeatable Read* (повторное чтение даст тот же результат, но возможен *phantom read*). *Serializable* (полная сериализация).
- Какие проблемы решает каждый уровень?
 - *Read Committed* — убираем *dirty read*. *Repeatable Read* — убираем *non-repeatable read*. *Serializable* — убираем *phantom read* и всё остальное.
- В Postgres какой уровень по умолчанию?
 - *Read Committed*. В Postgres вообще нет *Read Uncommitted* — это его особенность. *Repeatable Read* в Postgres ДОПОЛНИТЕЛЬНО защищает от *phantom read* через MVCC.
- Можно ли везде ставить Serializable?
 - Можно, но не нужно. *Serializable* — самый строгий уровень, и он сильно просаживает производительность: транзакции часто конфликтуют и откатываются. Для большинства операций хватает *Read Committed*.

► Нормализация

- Что такое нормализация и какие нормальные формы знаешь?
 - Нормализация — процесс приведения схемы БД к виду без избыточности данных. 1НФ — все атрибуты атомарны (нет списков в одной ячейке). 2НФ — 1НФ + каждый не-ключевой атрибут зависит от полного ключа. 3НФ — 2НФ + нет транзитивных зависимостей (не-ключевой атрибут не зависит от другого не-ключевого). На практике в проекте часто доходят до 3НФ, дальше идёт денормализация для производительности.

► VIEW и хранимые процедуры

- Что такое VIEW?
 - Сохранённый SELECT-запрос, к которому можно обращаться как к таблице. Виртуальная таблица — данных в ней нет, при обращении выполняется исходный запрос.
- В чём отличие MATERIALIZED VIEW?
 - Хранит результат запроса физически. Быстрее на чтение, но нужно периодически обновлять (REFRESH). Используется для тяжёлых аналитических запросов.
- Что такое триггер?
 - Хранимая процедура, которая автоматически вызывается БД при определённом событии (BEFORE / AFTER INSERT / UPDATE / DELETE). Часто используется для аудита, валидации, поддержки целостности. В банке любят триггеры для аудита.

► NoSQL базово

- Какие NoSQL ты знаешь?
 - *MongoDB* — документная (хранит JSON-подобные документы в коллекциях). *Redis* — *key-value*, в памяти, очень быстрая. Часто используется как кэш. *Cassandra* — *column-family*, для очень больших объёмов с записью.

- Что такое Master-Slave репликация?

→ Master принимает запись. Slave (replica) — копия, на которую идут чтения. Изменения с Master реплицируются на Slave. Помогает: (1) масштабировать чтение. (2) Резервная копия (если Master упал — promotion Slave в Master). (3) Аналитические запросы на Slave не мешают записи на Master.

► Способы работы с БД из Java

- Какие способы работы с БД в Java?

→ JDBC — низкий уровень, прямой SQL и работа с ResultSet. JPA / Hibernate — ORM, маппинг объектов на таблицы. jOOQ — typesafe SQL builder, generated code. Spring Data — удобная обёртка над JPA, query methods. Всё в итоге работает через JDBC-драйвер.

08 Spring и Spring Boot

На стажёре Spring спрашивают «на уровне знаю что это». Не ждут глубокого знания AOP, всех propagation, BeanPostProcessor'ов. Но базу — IoC / DI, бины, аннотации @Component / @Service / @Repository, @Autowired, как работает @Transactional — должен знать уверенно.

► Зачем нужен Spring

- Что такое Spring и зачем он нужен?

→ Фреймворк, который берёт на себя «инфраструктурный код»: создание объектов, связи между ними (DI), управление транзакциями, обработка HTTP, доступ к БД. Освобождает от boilerplate. Главная идея — IoC контейнер: фреймворк управляет жизненным циклом объектов, не разработчик.

► IoC и DI

- Что такое IoC?

→ Inversion of Control — инверсия управления. Вместо того чтобы класс сам создавал свои зависимости (new), он получает их извне — фреймворк сам решает, кого с кем связать.

- Что такое DI?

→ Dependency Injection — конкретная реализация IoC. Зависимости передаются объекту: через конструктор, сеттер или поле. ApplicationContext в Spring — это, по сути, Map<id, bean>. Spring сам создаёт бины и инжектит их друг в друга.

- Как создать бин в Spring?

→ (1) Аннотация на классе: @Component, @Service, @Repository, @Controller — Spring найдёт через component scan. (2) Метод с @Bean внутри @Configuration — явное создание (для бинов из чужих библиотек или сложной инициализации). (3) XML — устаревший способ, в новых проектах не используется.

- Чем отличается @Component, @Service, @Repository, @Controller?

→ Технически — все создают бин (синглтон по умолчанию). Семантически разные: @Repository — слой доступа к данным, Spring дополнительно оборачивает SQL/JPA исключения в DataAccessException. @Service — бизнес-логика. @Controller — web-слой. @Component — общий. Использование правильной аннотации улучшает читаемость кода.

► Способы инъекции

- Какие способы инъекции бинов есть?

→ Field (через @Autowired на поле) — короткий, но плохой. Setter (через @Autowired на сеттере) — гибкий, но позволяет создать неполностью сконфигурированный объект. Constructor — лучший: final-поля, видны все зависимости в сигнатуре, легко тестировать, не позволяет циклические зависимости.

- Почему constructor injection лучше field injection?

→ (1) Можно делать поля final — иммутабельность. (2) Видны все зависимости — если их 10, конструктор большой, это сигнал «класс делает слишком много» (SRP). (3) Легко тестировать без Spring — обычный new SomeService(mockA, mockB). (4) Невозможны циклические зависимости.

- Если поле не final при constructor injection — будет ошибка?

→ Нет. Spring отлично инжектит и в не-final поля. Но смысл конструктора частично теряется — поле потом можно перезаписать. Конвенция: всегда final.

► @Autowired

- Как работает @Autowired?

→ Spring находит бин подходящего типа в контексте и подставляет. Если в контексте несколько бинов одного типа — нужна @Qualifier или @Primary, иначе ошибка. С Spring 4.3+ @Autowired на единственном конструкторе можно не писать.

- Как разрешить конфликт, если бинов несколько?

→ @Primary — пометить «приоритетный» бин. @Qualifier("name") — точно выбрать конкретный. @Profile("prod") — активировать в определённом окружении. @ConditionalOnProperty / @ConditionalOnMissingBean — условное создание.

► Скоупы

- Какие скоупы бывают у бинов?

→ Singleton (default) — один экземпляр на контекст. Prototype — новый объект на каждый запрос. Request, Session, Application — для веб-приложений. Самое важное: Singleton не потокобезопасен сам по себе — если в нём mutable state, нужна синхронизация.

► Жизненный цикл бина

- Расскажи упрощённый жизненный цикл бина.

→ (1) Spring находит определение (BeanDefinition). (2) Вызывает конструктор. (3) Инъектит зависимости (setter / field). (4) @PostConstruct — пользовательская инициализация. (5) BeanPostProcessor.postProcessAfterInitialization — здесь создаются прокси (для @Transactional, AOP). (6) Бин готов и используется. (7) При закрытии контекста — @PreDestroy.

► @Transactional

- Что такое @Transactional?

→ Аннотация, которая открывает транзакцию вокруг метода. На успешный возврат — COMMIT, на RuntimeException — ROLLBACK. Реализуется через AOP-прокси: вызов идёт через прокси, который и управляет транзакцией.

- Что произойдёт при вызове @Transactional-метода из того же класса?

→ Транзакция НЕ откроется. Внутри одного бина вызов this.method() идёт напрямую на объект, в обход прокси. Это любимая ловушка на собеседах — самая частая ошибка. Решения: (1) Self-injection — внедрить бин сам в себя через ApplicationContext. (2) Вынести метод в другой бин. (3) Перейти на AspectJ — там работает self-invocation.

- Какие propagation знаешь?

→ REQUIRED (default) — использует существующую транзакцию или создаёт новую. REQUIRES_NEW — приостанавливает текущую и создаёт новую. Полезно для аудита: должно записаться даже если основная транзакция упала. На стажёре этих двух хватит, остальные (NESTED, MANDATORY, SUPPORTS, NEVER, NOT_SUPPORTED) знать как имена.

► Spring Boot

- В чём плюшка Spring Boot?

→ Автоконфигурация — Spring Boot сам подключает компоненты в зависимости от того, что есть на classpath. Стартеры — готовые наборы зависимостей (spring-boot-starter-web подтягивает Spring MVC, Tomcat, Jackson). Встроенный веб-сервер (Tomcat). Минимум конфигурации в XML, всё через свойства и аннотации.

- Что такое @SpringBootApplication?

→ Композиция трёх аннотаций: `@SpringBootConfiguration` (= `@Configuration`), `@EnableAutoConfiguration` (запуск автоконфигурации), `@ComponentScan` (сканирование пакетов начиная с этого класса).

- Что делает `@ComponentScan`?

→ Сканирует указанный пакет (по умолчанию — пакет класса, на котором стоит) и все подпакеты на предмет аннотаций `@Component`, `@Service`, `@Repository`, `@Controller`. Найденные классы регистрируются как бины в контексте.

- Что такое автоконфигурация?

→ Spring Boot имеет много готовых «конфигурационных рецептов». Они применяются автоматически, если выполнены условия: например, есть нужные классы на classpath. Так подключение `spring-boot-starter-data-jpa` автоматически настраивает `EntityManager`, `DataSource`, `TransactionManager`.

► **@RestController и Jakarta EE**

- Чем `@RestController` отличается от `@Controller`?

→ `@RestController` = `@Controller` + `@ResponseBody`. То есть результаты методов сразу сериализуются в JSON / XML (через Jackson) и отдаются в body ответа. `@Controller` сам по себе ожидает имя view (для рендера HTML), что для REST не нужно.

- Что такое Jakarta EE и чем отличается от Spring?

→ Jakarta EE (раньше Java EE) — стандарт корпоративной Java. Включает спецификации: сервлеты, JPA, JMS, EJB. Реализации — серверы приложений (GlassFish, WildFly). Spring — фреймворк поверх стандартной Java, не требует тяжёлого application-сервера. Сейчас Spring доминирует, Jakarta EE — в основном в legacy.

09 Hibernate / JPA (минимум)

На стажёре глубокого знания Hibernate не ждут. Достаточно: «что такое ORM», «JPA vs Hibernate», основные аннотации Entity, Lazy vs Eager, что такое N+1, что такое LazyInitializationException.

► JPA vs Hibernate

- В чём разница JPA и Hibernate?

→ JPA (Java Persistence API) — спецификация (интерфейсы и аннотации в пакете `javax.persistence` / `jakarta.persistence`). Hibernate — конкретная реализация. Альтернативы: EclipseLink, OpenJPA. Через JPA можно писать переносимый код, но в реальности почти всегда используют Hibernate-специфичные функции (например, `@JoinFormula`).

► ORM

- Что такое ORM?

→ Object-Relational Mapping — отображение объектов на таблицы БД. Класс `User` → таблица `users`, поля → колонки, связи (`@OneToMany`) → JOIN'ы. Hibernate берёт на себя SQL — разработчик работает с объектами. Плюс — меньше boilerplate, минус — нужно понимать, что Hibernate генерирует под капотом.

- Hibernate vs JDBC — в чём разница?

→ JDBC — низкий уровень, прямой SQL, `ResultSet`, ручной маппинг колонок в поля. Hibernate — поверх JDBC. Генерирует SQL автоматически, маппит результаты в объекты. Удобно для типового CRUD, но JDBC даёт полный контроль для сложных запросов.

► Аннотации Entity

- `@Entity` — класс является сущностью JPA.
- `@Table(name = "users")` — указать имя таблицы (если отличается от класса).
- `@Id` — первичный ключ.
- `@GeneratedValue(strategy = GenerationType.IDENTITY)` — автогенерация ключа.
- `@Column(name = "...", nullable = false, length = 100)` — настройка колонки.
- `@Transient` — поле не сохраняется в БД.

► Связи

- Какие виды связей бывают?

→ `@OneToOne` — один к одному (`User ↔ Passport`). `@OneToMany` / `@ManyToOne` — один ко многим / многие к одному (`User → Orders`). `@ManyToMany` — многие ко многим (`Student ↔ Course`, через промежуточную таблицу).

- Какой Fetch type по умолчанию?

→ Для коллекций (`@OneToMany`, `@ManyToMany`) — `Lazy`. Для скалярных связей (`@OneToOne`, `@ManyToOne`) — `Eager`. `Eager` по умолчанию для `@ManyToOne` — частая ловушка: при загрузке `Order` сразу подтянется `User`, а если у `User` тоже есть `@ManyToOne` — и его связь. Может уйти каскадом.

► Lazy vs Eager

- Что лучше — Lazy или Eager?

→ *Lazy. Eager почти всегда плохо — Hibernate тянет лишнее, и часто это даёт N+1 в неожиданном месте. Правило: ставь Lazy везде, а конкретный JOIN FETCH делай в запросах, где данные действительно нужны.*

- **Что такое LazyInitializationException?**

→ *Возникает, когда обращаешься к LAZY-полю после закрытия сессии Hibernate. Сценарий: достал Entity в @Transactional-методе, вернул наружу. Транзакция закрылась — сессия закрылась. В контроллере / сериализаторе обращаешься к LAZY-полю — БАХ. Решения: (1) @EntityGraph на репозиторий — указать, что грузить eager. (2) JOIN FETCH в JPQL. (3) DTO-проекция в сервисе. (4) Open Session In View — но это плохая практика.*

- ▶ **N+1 проблема**

- **Что такое N+1?**

→ *Загрузил список из N сущностей одним запросом. Потом в цикле обращаешься к их LAZY-связям — на каждую идёт отдельный SELECT. Итого 1 + N запросов. На 100 сущностях это 101 запрос — катастрофа для производительности.*

- **Как решить N+1?**

→ *(1) JPQL с JOIN FETCH: select o from Order o join fetch o.user. (2) @EntityGraph — атрибут на репозиторий-методе, говорит «подгрузить эти поля сразу». (3) Hibernate batch_size — группирует SELECT'ы по пачкам. (4) DTO-проекция через JPQL select new com.x.OrderDto(...) — сразу плоский результат без загрузки entity.*

- ▶ **Spring Data**

- **Что такое Spring Data JPA?**

→ *Удобная обёртка над JPA / Hibernate. Создаёшь интерфейс, наследуешь JpaRepository — Spring сам сгенерирует реализацию. Query methods: метод findByEmail — Spring сам соберёт SQL по имени. @Query — кастомный JPQL или native SQL. Pageable — пагинация и сортировка.*

```
public interface UserRepository extends JpaRepository<User, Long> {  
    // Метод сам генерируется по имени  
    Optional<User> findByEmail(String email);  
    List<User> findByAgeGreaterThanOrderByAsc(int age);  
  
    // Кастомный запрос  
    @Query("select u from User u where u.active = true and u.balance > :min")  
    List<User> findActiveWithBalance(@Param("min") BigDecimal min);  
}
```

10 HTTP и REST

На стажёре HTTP спрашивают обязательно. Структура запроса, методы, идемпотентность, статус-коды, REST vs SOAP. Глубокого знания не ждут, но базу должны быть на слух.

► Структура HTTP-запроса

- Из чего состоит HTTP-запрос?

→ *Стартовая строка: метод + URL + версия (GET /users HTTP/1.1). Headers (Content-Type, Accept, Authorization). Пустая строка. Body (для POST/PUT/PATCH). Ответ — статус + версия + headers + body.*

► HTTP-методы

- Какие HTTP-методы знаешь?

→ *GET — получение данных, тело не предполагается. POST — создание, отправка данных в body. PUT — полное обновление / создание. PATCH — частичное обновление. DELETE — удаление. HEAD — как GET, но без body (только headers). OPTIONS — какие методы разрешены (CORS).*

- Что такое идемпотентность?

→ *Многokrатный вызов даёт тот же результат, что и однократный. Не путать с «безопасностью» (никак не меняет состояние) — GET и safe, и идемпотентен. PUT и DELETE идемпотентны (повторное PUT того же — тот же результат), но НЕ safe (меняют состояние). POST НЕ идемпотентен (создаст две записи). PATCH обычно тоже не идемпотентен.*

- В чём разница PUT и PATCH?

→ *PUT — полная замена ресурса. Передаём весь объект целиком. Если не указать поле — оно станет null. PATCH — частичное обновление. Передаём только изменяемые поля, остальное остаётся.*

► Статус-коды

- Какие группы статус-кодов есть?

→ *2xx — успех (200 OK, 201 Created, 204 No Content). 3xx — редирект (301 постоянный, 302 временный, 304 Not Modified). 4xx — ошибка клиента (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable Entity, 429 Too Many Requests). 5xx — ошибка сервера (500 Internal, 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout).*

- В чём разница 401 и 403?

→ *401 Unauthorized — НЕ АУТЕНТИФИЦИРОВАН (не знаем кто ты — нет токена / он невалидный). Сделай login и приходи снова. 403 Forbidden — АУТЕНТИФИЦИРОВАН, но нет ПРАВ. Знаем кто ты, но эту операцию делать не можешь. Простая ловушка на собеседах — часто путают.*

► Headers

- Content-Type — тип тела запроса/ответа (application/json, application/xml, text/plain).
- Accept — какой формат ответа клиент хочет.
- Authorization — токен или Basic Auth.
- Cache-Control, ETag — для кэширования.
- Кастомные X-* (X-Request-Id, X-Trace-Id) — для трассировки.

► REST

- Что такое REST?

- Архитектурный стиль построения веб-сервисов. Идея: ресурсы (нечто, что имеет URL), действия над ними (через HTTP-методы), без сохранения состояния на сервере. Не протокол, а набор принципов.

- Какие принципы у REST?

- *Stateless* — сервер не хранит сессию клиента, каждый запрос самодостаточен. *Cacheable* — ответы могут кэшироваться. *Uniform interface* — единый интерфейс через HTTP-методы и URL. *Layered system* — между клиентом и сервером могут быть прокси, балансеры. *Client-Server* — клиент и сервер независимы. *HATEOAS* — клиент находит переходы через ссылки в ответах (на практике редко делают).

- Почему REST stateless?

- Без сессии на сервере любой запрос можно отправить на любой сервер кластера. Это позволяет легко горизонтально масштабироваться: добавил инстанс — балансер начал слать на него. Состояние держит клиент (например, в JWT).

► REST vs SOAP

- В чём разница REST и SOAP?

- REST — архитектурный стиль на HTTP. Обычно JSON. Гибкий, контракт через OpenAPI. SOAP — протокол на XML с собственным envelope (Header + Body). Строгий контракт через WSDL. Поддерживает WS-Security (подпись XML, шифрование). В банках живёт в legacy-интеграциях (с ЦБ, межбанком, SWIFT).

- JSON vs XML — в чём разница?

- JSON компактнее, читабельнее, нативно поддерживается в JavaScript. XML — стандартизированный, есть схемы (XSD), namespaces, можно валидировать. Для REST почти всегда JSON, для SOAP — обязательно XML.

- Можно ли в GET передать body?

- Спецификация не запрещает, но рекомендует не использовать. Многие фреймворки и прокси игнорируют body в GET. Если параметров много — лучше POST с явной семантикой («поиск с телом») или query string.

11 Git, Maven, Liquibase

Базовые инструменты. На стажёре спросят 2–3 команды Git, что такое pom.xml, что такое Pull Request. Глубокие вопросы (rebase vs merge, конфликты) могут быть, если ты сам начнёшь.

► Git

- Какие команды Git знаешь?

→ *git clone* — скачать репозиторий. *git pull* — забрать изменения. *git push* — отправить. *git add* — добавить файлы в *staging*. *git commit* — закоммитить. *git branch* — создать / посмотреть ветки. *git checkout* — переключить ветку. *git merge* — слить ветку. *git rebase* — переписать историю поверх другой ветки. *git stash* — отложить незакоммиченные изменения. *git status* / *git log* / *git diff* — состояние.

- В чём разница merge и rebase?

→ *merge* сохраняет историю как есть — создаёт *merge*-коммит, объединяющий ветки. Виден факт слияния. *rebase* «перематывает» твои коммиты поверх свежей основной ветки — история линейная, без *merge*-коммитов. *rebase* нельзя делать на публичных ветках (*master*), которые уже у кого-то стоят — переписывание истории ломает им работу.

- Что такое Pull Request / Merge Request?

→ Предложение влить твою ветку в основную (*master* / *main* / *develop*). Коллеги делают код-ревью, оставляют комментарии, требуют изменений. После одобрения — *merge*. В банках обычно нужно 2 аппрува.

- Как разрешать конфликты при merge?

→ Git помечает конфликтные участки в файле символами <<<<<<, =====, >>>>>>. Вручную решаешь, что оставить, удаляешь маркеры, делаешь *git add* и *git commit*. Современные IDE (IntelliJ) дают удобный визуальный *merge-tool*.

- Что такое .gitignore?

→ Файл со списком путей и шаблонов, которые Git должен игнорировать. Туда обычно кладут: *target/* (Maven), *node_modules/*, *.idea/* (IntelliJ), **.log*, файлы с секретами (хотя секреты лучше вообще не класть в репозиторий).

- Что такое Git Flow и Trunk-based?

→ *Git Flow* — модель с долгоживущими ветками *master*, *develop*, *feature/**, *release/**, *hotfix/**. Подходит для релизного цикла с явными версиями. *Trunk-based* — все работают близко к *master* (через короткие *feature-branch*'ы), часто мерджат. Используется в командах с CI/CD и непрерывным деплоем.

► Maven

- Что такое pom.xml?

→ *Project Object Model* — файл конфигурации Maven-проекта. Содержит: координаты артефакта (*groupId*, *artifactId*, *version*), зависимости, плагины, свойства, профили. Это сердце Maven-проекта.

- Расскажи жизненный цикл Maven.

→ *validate* → *compile* → *test* → *package* → *verify* → *install* → *deploy*. *validate* — проверка проекта. *compile* — компиляция. *test* — юнит-тесты. *package* — упаковка в *jar/war*. *verify* — интеграционные тесты. *install* — в локальный репозиторий *~/m2*. *deploy* — в удалённый репозиторий (Nexus, Artifactory).

- Какие scope у зависимостей в Maven?

→ *compile* (default) — доступно везде, идёт в финальный *jar*. *test* — только для тестов. *provided* — для компиляции, но не в финальный *jar* (например, *servlet-api* — его даёт Tomcat). *runtime* — в

рантайме, но не при компиляции (например, JDBC-драйвер). `system` — локальный `jar`, не использовать.

- Что делает `mvn clean install`?

→ `clean` — удаляет директорию `target/`. `install` — проходит весь жизненный цикл (`compile` → `test` → `package`) и кладёт артефакт в локальный репозиторий `~/m2`. Самая часто используемая команда.

- Maven vs Gradle?

→ `Maven` — XML-декларатив (`pom.xml`), строгая структура, проще. `Gradle` — `Groovy` / `Kotlin DSL`, гибче, быстрее (`incremental builds`). В Сбере и большинстве банков — `Maven`, в `Android` и многих стартапах — `Gradle`.

► Liquibase / Flyway

- Зачем нужны Liquibase или Flyway?

→ Управление версиями схемы БД. Без них: разработчики применяют SQL руками, кто-то применил, кто-то нет, на проде катастрофа. С `Liquibase`: `changeset`-ы (изменения) лежат в репозитории, при старте приложения автоматически применяются нужные. Каждый `changeset` запускается один раз и фиксируется в специальной таблице `DATABASECHANGELOG`.

- Как работает Liquibase?

→ (1) Описываешь `changeset`'ы — изменения схемы. Формат: XML, YAML, JSON или SQL. (2) При старте `Liquibase` подключается к БД, смотрит в таблицу `DATABASECHANGELOG` — какие уже применены. (3) Применяет новые `changeset`'ы. (4) Записывает факт применения. Поддерживает `rollback` — можно откатить `changeset`.

- Liquibase vs Flyway?

→ `Liquibase` — описание изменений в XML / YAML, поддерживает `rollback`, кросс-БД (одно описание — для разных БД). `Flyway` — версионированные SQL-скрипты (`V1__init.sql`, `V2__add_table.sql`), проще, но без `rollback` и кросс-БД. Сбер любит `Liquibase`, многие стартапы — `Flyway`.

12 Паттерны проектирования

На Сбере паттерны спрашивают почти всегда. От стажёра не ждут знания всех 23 паттернов GoF, но базовые — Singleton, Builder, Factory, Observer, Strategy, Proxy vs Decorator — должны быть на слух с примерами.

► Три группы паттернов

- На какие три группы делятся паттерны?

→ *Порождающие (Creational) — про создание объектов: Singleton, Builder, Factory Method, Abstract Factory, Prototype. Структурные (Structural) — про композицию объектов: Adapter, Decorator, Proxy, Facade, Composite, Bridge. Поведенческие (Behavioral) — про взаимодействие: Observer, Strategy, Chain of Responsibility, Iterator, Command, State, Template Method.*

► Singleton

- Что такое Singleton?

→ *Гарантирует, что в системе будет один экземпляр класса, и даёт глобальную точку доступа. Используется для конфигов, логгеров, пулов соединений. В Spring каждый @Component по умолчанию — Singleton.*

- Как реализовать Singleton?

→ *Самый простой и правильный способ — через enum (с Java 5):*

```
public enum Database {  
    INSTANCE;  
    public void connect() { /* ... */ }  
}  
  
// Использование  
Database.INSTANCE.connect();
```

Альтернативы — через приватный конструктор + статический метод с double-checked locking. Но enum проще и потокобезопасен из коробки.

► Builder

- Когда нужен Builder?

→ *Когда у объекта много полей (особенно опциональных), и не хочется делать конструктор с 10 параметрами. Builder делает создание объекта пошаговым и читаемым.*

```
User user = User.builder()  
    .name("Иван")  
    .email("ivan@example.com")  
    .age(25)  
    .build();
```

В Java часто используют Lombok с @Builder, чтобы не писать руками. С Java 14+ Records частично заменяют простые Builder-ы.

► Factory Method и Abstract Factory

- В чём разница Factory Method и Abstract Factory?

→ *Factory Method* — один метод, создающий объект. Часто используется для создания разных подклассов одного типа. Например, `createConnection()` возвращает `PostgresConnection` или `OracleConnection`. *Abstract Factory* — фабрика фабрик. Создаёт связанные семейства объектов. Например, GUI-фабрика для разных ОС, создающая совместимые между собой кнопки, окна, поля.

► Observer (Publisher-Subscriber)

▪ Что такое Observer?

→ *Поведенческий паттерн*: один объект (*Subject*) хранит список подписчиков (*Observer*), при изменении состояния — уведомляет всех. В Java встречается: `PropertyChangeListener`, `java.util.Observer` (устарел), `Spring Events`, `RxJava`. Все системы событий — это *Observer*.

```
interface EventListener {
    void onEvent(Event e);
}

class EventBus {
    private final List<EventListener> listeners = new ArrayList<>();
    public void subscribe(EventListener l) { listeners.add(l); }
    public void publish(Event e) {
        for (EventListener l : listeners) l.onEvent(e);
    }
}
```

► Strategy

▪ Что такое Strategy?

→ *Поведенческий паттерн*: вынесение алгоритма в отдельный класс. У клиента — поле типа интерфейса, можно подменять реализацию. Например, разные алгоритмы сортировки, разные способы расчёта комиссии. Хорошая замена `switch-case` с типами.

► Proxy vs Decorator (любимый вопрос Сбера)

▪ В чём разница Proxy и Decorator?

→ Оба — обёртка над другим объектом, реализуют тот же интерфейс. Но цели разные: *Proxy* — НЕ МЕНЯЕТ поведение оригинального объекта. Добавляет «инфраструктуру»: контроль доступа, ленивая инициализация, кэширование, логирование. Используется в `@Transactional Spring`. *Decorator* — ДОБАВЛЯЕТ функционал. Можно навешивать слоями: `BufferedInputStream(GZIPInputStream(FileInputStream))` — каждый слой добавляет функциональность.

► DTO, POJO, VO, Entity

▪ Что такое POJO?

→ *Plain Old Java Object* — обычный Java-класс без специальных требований (наследования от чего-то, реализации интерфейсов фреймворка). Просто поля + геттеры + сеттеры + конструктор.

▪ Что такое DTO?

→ *Data Transfer Object* — объект для передачи данных между слоями системы или между сервисами. Не содержит логики, только поля и геттеры/сеттеры. Часто используется как тело REST-запросов и ответов.

▪ В чём разница DTO и Entity?

→ *Entity* — связана с БД (`@Entity`). Может иметь связи (`LazyInitializationException` наружу!). *DTO* — независимый объект для передачи. Хорошая практика: НЕ возвращать *Entity* из REST-контроллеров, маппить в *DTO*. Это разрывает зависимость API от схемы БД.

- Что такое VO?

→ *Value Object* — объект, который определяется ТОЛЬКО своими значениями (не *identity*).
Например, *Money(amount, currency)* — два VO с одинаковыми полями равны. Иммутабельны, имеют *equals/hashCode* по содержимому. С Java 14+ удобно делать через *record*.

13 Тесты — JUnit и Mockito

На стажёре спрашивают: зачем вообще тесты, какие виды бывают, базовые аннотации JUnit, базовое Mockito (что такое мок). Если в учебном проекте писал тесты — обязательно упомяни.

► Зачем тесты

▪ Зачем писать тесты?

→ (1) Защита от регресса — поменял код, прогнал тесты, увидел что сломалось. (2) Документация — тест показывает, как код должен использоваться. (3) Дизайн — код, который сложно тестировать, обычно плохо спроектирован. (4) Уверенность при рефакторинге — можно смело менять, тесты подскажут.

► Виды тестов

▪ Какие виды тестов бывают?

→ Unit (юнит) — проверяет один класс или метод изолированно, зависимости замокированы. Быстро, много, легко отлаживать. Integration — проверяет взаимодействие нескольких компонентов, обычно с реальной БД (через Testcontainers). E2E (end-to-end) — проверяет систему целиком через UI или API, имитирует поведение пользователя. Медленно, мало, дорого.

► JUnit 5

▪ Какие базовые аннотации JUnit 5?

→ @Test — обозначает тестовый метод. @BeforeEach / @AfterEach — выполняется перед/после каждого теста. @BeforeAll / @AfterAll — один раз перед/после всех тестов класса (метод должен быть static). @DisplayName — человекочитаемое имя теста. @ParameterizedTest — запуск теста с разными входными параметрами. @Disabled — отключить.

▪ Какие assertions используются?

→ assertEquals(expected, actual) — равенство. assertTrue / assertFalse — для boolean. assertNotNull / assertNull. assertThrows(Exception.class, () -> ...) — проверка, что код бросает исключение. AssertJ (отдельная либа) даёт fluent API: assertThat(value).isEqualTo(...).isNotNull().

```
@Test
@DisplayName("Деление на ноль бросает ArithmeticException")
void divideByZero() {
    Calculator c = new Calculator();
    assertThrows(ArithmeticException.class, () -> c.divide(10, 0));
}
```

► Mockito

▪ Что такое Mockito?

→ Библиотека для создания моков (имитаций) объектов в тестах. Позволяет «подменить» зависимость тестируемого класса фейком, который ведёт себя так, как тебе нужно.

▪ Базовый набор Mockito?

→ @Mock — создать мок. @InjectMocks — создать тестируемый объект и подставить в него моки. @ExtendWith(MockitoExtension.class) — активировать. when(mock.method(...)).thenReturn(...) — задать поведение. verify(mock).method(...) — проверить, что метод был вызван. ArgumentCaptor — захватить аргументы, с которыми звали мок.

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest {
```

```

@Mock UserRepository repo;
@InjectMocks UserService service;

@Test
void findByIdReturnsName() {
    when(repo.findById(1L)).thenReturn(Optional.of(new User(1L, "Иван")));

    String name = service.getName(1L);

    assertEquals("Иван", name);
    verify(repo).findById(1L);
}
}

```

- В чём разница Mock и Spy?

→ *Mock* — полная имитация, по умолчанию все методы возвращают null / 0 / false. Поведение задаёшь через `when().thenReturn()`. *Spy* — обёртка над реальным объектом. Методы, которые ты не подменил, работают как обычно. Полезно, когда хочется подменить ОДИН метод реального класса, не переписывая весь.

► Spring Boot тесты

- Какие аннотации для тестов в Spring Boot?

→ *@SpringBootTest* — поднимает весь контекст приложения. Долго, но полноценно. *@WebMvcTest* — только web-слой (контроллеры), без БД. *@DataJpaTest* — только JPA-слой, с in-memory БД. *MockMvc* — имитация HTTP-запросов в тестах.

14 Алгоритмы и структуры данных

Сбер прямо подчёркивает: алгоритмы и структуры данных проверяют. Глубоких LeetCode Hard не ждут — задачи Easy. Но Big-O знать обязательно, базовые сортировки, бинарный поиск, основные структуры. Этот раздел — теория. Сами задачи с разбором — в разделе 15.

► Big-O нотация

- Что такое Big-O?

→ Способ описать асимптотическую сложность алгоритма — как растёт время выполнения с ростом входа. $O(1)$ — константа, не зависит от размера. $O(\log n)$ — логарифм (бинарный поиск). $O(n)$ — линейная (один проход). $O(n \log n)$ — хорошие сортировки (Timsort, merge sort). $O(n^2)$ — квадратичная (пузырёк). $O(2^n)$ — экспоненциальная (наивная фибоначчи через рекурсию).

- Какая сложность у `HashMap.get`?

→ $O(1)$ в среднем. $O(\log n)$ худший случай при treeified бакете (с Java 8). $O(n)$ при катастрофически плохом hashCode.

- Какая сложность у бинарного поиска?

→ $O(\log n)$. Каждая итерация уменьшает область поиска вдвое.

► Базовые структуры

- Что такое стек и где применяется?

→ LIFO (Last In, First Out). Используется в: (1) вызовы функций в JVM (stack frames). (2) Undo / отмена операций. (3) Парсинг (проверка корректности скобок). (4) Обход дерева в глубину (DFS). В Java — Deque (предпочтительно) или Stack (legacy).

- Что такое очередь и где применяется?

→ FIFO (First In, First Out). Используется в: (1) Обход графа в ширину (BFS). (2) Планировщик задач. (3) Producer-consumer (BlockingQueue). В Java — LinkedList, ArrayDeque, или ConcurrentLinkedQueue для многопоточности.

- Что такое дек?

→ Двусторонняя очередь. Можно добавлять и удалять с обоих концов. В Java — Deque interface, реализация ArrayDeque. Универсальная — может работать как стек и как очередь.

► Деревья

- Что такое бинарное дерево поиска (BST)?

→ Дерево, где для каждого узла: все элементы в левом поддереве меньше, в правом — больше. Поиск, вставка, удаление — $O(\log n)$ в сбалансированном дереве. В несбалансированном — деградирует до $O(n)$ (как связный список).

- Что такое AVL и красно-чёрное дерево?

→ Самобалансирующиеся BST. После каждой вставки/удаления выполняется ребалансировка через повороты — гарантия $O(\log n)$. AVL — строже балансируется (быстрее поиск, медленнее вставка). Красно-чёрное — менее строго (используется в TreeMap, TreeSet).

► Hash-таблица

- Как устроена хэш-таблица и как разрешаются коллизии?

→ Массив бакетов. По хэшу ключа определяется индекс. Две стратегии разрешения коллизий: (1) Chaining (цепочки) — в бакете связный список. Java HashMap использует это (с Java 8 — список

превращается в дерево при ≥ 8 элементов). (2) Open addressing — если бакет занят, ищем следующий свободный. Используется в некоторых других языках.

► Сортировки

- Какие сортировки знаешь?

→ Простые $O(n^2)$: пузырьрёк (bubble), выбор (selection), вставка (insertion). Эффективные $O(n \log n)$: merge sort (через слияние), quick sort (через разбиение), heap sort (через кучу). Специфичные: counting sort $O(n+k)$, radix sort $O(n \times d)$ — для целых чисел.

- Какую сортировку использует Java по умолчанию?

→ Для массивов примитивов — Dual-Pivot Quicksort. Для объектов — Timsort (гибрид merge и insertion sort, разработан для Python, перенесён в Java). Timsort стабилен и эффективен на реальных данных, где часто есть упорядоченные подпоследовательности.

► Бинарный поиск

- Как работает бинарный поиск?

→ Только для отсортированного массива. Берём середину, сравниваем с искомым. Если меньше — ищем в левой половине, больше — в правой. Каждая итерация — $\div 2$. Сложность $O(\log n)$.

ВНИМАНИЕ • Подводный камень бинарного поиска

Наивный код: $\text{mid} = (\text{left} + \text{right}) / 2$ — может переполниться, если left и right большие int.

Правильно: $\text{mid} = \text{left} + (\text{right} - \text{left}) / 2$ — переполнения не будет.

Сбер любит этот вопрос на собеседовании, обязательно знать.

```
public static int binarySearch(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2; // защита от overflow
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

► BFS и DFS

- Что такое BFS и DFS?

→ BFS (Breadth-First Search) — обход в ширину. Идёт по уровням. Используется в: поиск кратчайшего пути в невзвешенном графе, обход дерева по уровням. Внутри — очередь. DFS (Depth-First Search) — обход в глубину. Идёт вглубь до конца, потом откатывается. Используется в: поиск цикла, топологическая сортировка, проверка связности. Внутри — стек или рекурсия.

► Жадные алгоритмы

- Что такое жадный алгоритм?

→ На каждом шаге делаешь локально оптимальный выбор, надеясь, что в итоге получится глобально оптимальное решение. Работает не всегда! Пример где работает: размен сдачи стандартными монетами. Пример где НЕ работает: размен сдачи нестандартным набором (1, 3, 4) — для 6 жадный даст $4+1+1$, оптимально $3+3$.

► Two Pointers и Sliding Window

- Что такое метод двух указателей?

→ Используем два индекса, движущиеся по массиву по правилам. Часто для отсортированного массива: пара с заданной суммой, разворот строки, проверка палиндрома. Сложность обычно $O(n)$.

- Что такое скользящее окно?

→ Поддерживаем «окно» — диапазон $[left, right]$ в массиве. Двигаем $right$ (расширяем), при нарушении условия — двигаем $left$ (сужаем). Используется для подстрок без повторов, максимума в окне фиксированного размера. Сложность $O(n)$.

► Префиксные суммы

- Что такое префиксная сумма?

→ Массив, где $prefix[i] = sum(arr[0..i])$. Позволяет за $O(1)$ узнать сумму на отрезке $[l..r]$: $sum(l..r) = prefix[r] - prefix[l-1]$. Полезно когда много запросов сумм на разных отрезках одного массива.

15 Практические задачи (live coding)

Сбер практически всегда даёт live coding на технической части — 15–30 минут на задачу. Уровень Easy с LeetCode. Здесь — реальные задачи с собеседований Сбера (с пометкой номеров) и классические задачи Junior-уровня с разбором решений и объяснениями.

ФИШКА · Главный совет по live coding

- (1) Сначала проговори условие своими словами. Если что-то непонятно — задай уточняющий вопрос. На пустой массив что возвращать? Можно ли мутировать вход?
- (2) Расскажи план решения ДО написания кода.
- (3) Думай вслух. Молчание в течение минуты — плохой сигнал.
- (4) Тестируй на пограничных случаях: пустой вход, один элемент, null, очень большие числа. Интервьюер оценивает мыслительный процесс, а не только финальный код.

► Задача 1. Палиндром (Sber #65, март 2025)

«Проверить, является ли строка палиндромом — то есть читается одинаково в обе стороны». Простая, но Сбер любит её для разогрева. Что важно показать — обсуждение граничных случаев (пустая строка, 1 символ, null), знание сложности.

Ленивое решение

```
public static boolean isPalindromeLazy(String s) {
    if (s == null) return false;
    return new StringBuilder(s).reverse().toString().equals(s);
}
```

Работает, но создаёт лишний объект StringBuilder и копию строки. На собеседовании можно начать с него и сказать: «работает, но можно эффективнее».

Оптимальное решение (Сбер оценил как «правильное»)

```
public static boolean isPalindrome(String s) {
    if (s == null) return false;
    int l = 0, r = s.length() - 1;
    while (l < r) {
        if (s.charAt(l) != s.charAt(r)) return false;
        l++; r--;
    }
    return true;
}
```

Два указателя с концов. Сложность $O(n/2) = O(n)$ по времени, $O(1)$ по памяти. Это правильный ответ.

► Задача 2. Реализовать ArrayList с нуля (Sber #134 — стажировка!)

«Напиши класс MyArrayList<T> с методами add(element), get(index), remove(index)». Реальная задача со стажировки. Сбер любит давать её и просить отладить в реальном времени, найти баг.

```
public class MyArrayList<T> {
    private Object[] array;
    private int size = 0;
    private static final int DEFAULT_CAPACITY = 10;
}
```

```

public MyArrayList() {
    this.array = new Object[DEFAULT_CAPACITY];
}

public void add(T element) {
    if (size >= array.length) {        // ВНИМАНИЕ к условию!
        grow();
    }
    array[size++] = element;
}

@SuppressWarnings("unchecked")
public T get(int index) {
    if (index < 0 || index >= size) {
        throw new IndexOutOfBoundsException("Index: " + index + ", Size: " + size);
    }
    return (T) array[index];
}

public void remove(int index) {
    if (index < 0 || index >= size) {
        throw new IndexOutOfBoundsException("Index: " + index + ", Size: " + size);
    }
    // Сдвигаем элементы справа от index на 1 влево
    System.arraycopy(array, index + 1, array, index, size - index - 1);
    array[--size] = null; // помогаем GC
}

private void grow() {
    int newCapacity = array.length + (array.length >> 1); // 1.5x
    array = Arrays.copyOf(array, newCapacity);
}

public int size() { return size; }
}

```

ЛОВУШКА • Off-by-one в условии расширения

Самая частая ошибка: написать `if (size > array.length)` вместо `if (size >= array.length)`. Тогда при заполнении массива `add` потеряет элемент или выйдет за границу.

Сбер любит «найди баг»: специально просит запустить и проверить с числом элементов = `capacity`. Если кандидат не пробует пограничный случай — минус.

► Задача 3. Удалить анаграммы из списка (Sber #68, март 2025)

«Дан список строк. Убрать анаграммы (слова, состоящие из одних и тех же букв), оставить только уникальные».

Идея: две строки — анаграммы, если их отсортированные `char`-массивы равны. Используем `Map`, где ключ — отсортированная версия слова, значение — само слово.

```

public static List<String> removeAnagrams(List<String> words) {
    Map<String, String> unique = new LinkedHashMap<>();
    for (String word : words) {
        char[] chars = word.toCharArray();
        Arrays.sort(chars);
        String key = new String(chars);
        unique.putIfAbsent(key, word); // оставляем первое вхождение
    }
}

```

```
return new ArrayList<>(unique.values());
}
```

Сложность: $O(n \times k \log k)$, где n — число слов, k — средняя длина слова. LinkedHashMap нужен, чтобы порядок результата соответствовал порядку входа.

► Задача 4. Первый неповторяющийся символ (Sber, классика)

«Дана строка. Вернуть первый символ, который встречается только один раз».

Идея: два прохода. Первый — построить `Map<Character, Integer>` со счётчиками. Второй — пройти строку и вернуть первый символ с `count = 1`.

```
public static Character firstUnique(String s) {
    if (s == null || s.isEmpty()) return null;

    Map<Character, Integer> count = new HashMap<>();
    for (char c : s.toCharArray()) {
        count.merge(c, 1, Integer::sum);
    }

    for (char c : s.toCharArray()) {
        if (count.get(c) == 1) return c;
    }
    return null;
}
```

Сложность: $O(n)$ по времени, $O(k)$ по памяти, где k — размер алфавита. На практике $k \leq 256$ для ASCII, поэтому фактически $O(1)$ памяти.

► Задача 5. Развернуть строку без `StringBuilder.reverse`

```
public static String reverse(String s) {
    if (s == null) return null;
    char[] chars = s.toCharArray();
    int l = 0, r = chars.length - 1;
    while (l < r) {
        char tmp = chars[l];
        chars[l] = chars[r];
        chars[r] = tmp;
        l++; r--;
    }
    return new String(chars);
}
```

ФИШКА • Подвох с эмодзи

Если интервьюер спросит «а как с эмодзи?», правильный ответ: стандартный разворот `char`-массива сломает суррогатные пары (эмодзи 🔥 это два `char`).

Корректно для Unicode: преобразовать в `codePoints` (`int`), развернуть массив `int`, собрать обратно. На стажёре достаточно знать про существование проблемы.

► Задача 6. Парсинг строки `key=value;key2=value2`

```
public static Map<String, String> parse(String s) {
    Map<String, String> result = new HashMap<>();
    if (s == null || s.isEmpty()) return result;
```

```

    for (String pair : s.split(";")) {
        int eqIdx = pair.indexOf('=');
        if (eqIdx == -1) continue; // невалидная пара – пропускаем
        String key = pair.substring(0, eqIdx).trim();
        String val = pair.substring(eqIdx + 1).trim();
        result.put(key, val);
    }
    return result;
}

```

Не используем `split('=')`, потому что значение само может содержать `=`. Используем `indexOf` чтобы найти первый знак равенства.

► Задача 7. FizzBuzz

«Выведи числа от 1 до 100. Если делится на 3 — Fizz, на 5 — Buzz, на 15 — FizzBuzz».

```

public static void fizzBuzz(int n) {
    for (int i = 1; i <= n; i++) {
        if (i % 15 == 0) System.out.println("FizzBuzz");
        else if (i % 3 == 0) System.out.println("Fizz");
        else if (i % 5 == 0) System.out.println("Buzz");
        else System.out.println(i);
    }
}

```

ВНИМАНИЕ · Главная ловушка FizzBuzz

Проверка на 15 ДОЛЖНА быть первой. Если поставить `if (i % 3 == 0)` сначала — число 15 даст «Fizz» и упустит «FizzBuzz».

Альтернатива: одна проверка с конкатенацией: `if (i % 3 == 0) s += "Fizz"; if (i % 5 == 0) s += "Buzz";` — порядок не важен.

► Задача 8. Two Sum

«Дан массив и `target`. Найти индексы двух элементов с суммой = `target`».

Наивно — два цикла, $O(n^2)$. Эффективно — один проход с `HashMap`, $O(n)$.

```

public static int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> seen = new HashMap<>(); // value -> index
    for (int i = 0; i < nums.length; i++) {
        int need = target - nums[i];
        if (seen.containsKey(need)) {
            return new int[]{seen.get(need), i};
        }
        seen.put(nums[i], i);
    }
    throw new IllegalArgumentException("No two sum found");
}

```

► Задача 9. Найти дубли в массиве

```

public static Set<Integer> findDuplicates(int[] arr) {
    Set<Integer> seen = new HashSet<>();
    Set<Integer> duplicates = new HashSet<>();
    for (int x : arr) {
        if (!seen.add(x)) { // add возвращает false, если был
            duplicates.add(x);
        }
    }
}

```

```

    }
    return duplicates;
}

```

Сложность $O(n)$ по времени, $O(n)$ по памяти. Альтернатива через Stream:

```

Map<Integer, Long> counts = Arrays.stream(arr).boxed()
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
Set<Integer> dups = counts.entrySet().stream()
    .filter(e -> e.getValue() > 1)
    .map(Map.Entry::getKey)
    .collect(Collectors.toSet());

```

► Задача 10. Корректность скобок (Sber #55, ОТП, классика)

«Дана строка с (и). Проверь, что скобки сбалансированы».

Простой случай — только круглые скобки. Используем счётчик.

```

public static boolean isBalancedSimple(String s) {
    int balance = 0;
    for (char c : s.toCharArray()) {
        if (c == '(') balance++;
        else if (c == ')') {
            balance--;
            if (balance < 0) return false; // закрывающая без открывающей
        }
    }
    return balance == 0;
}

```

Усложнение: разные виды скобок {}[]

```

public static boolean isBalanced(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    Map<Character, Character> pairs = Map.of(')', '(', ']', '[' , '}', '{');

    for (char c : s.toCharArray()) {
        if (c == '(' || c == '[' || c == '{') {
            stack.push(c);
        } else if (pairs.containsKey(c)) {
            if (stack.isEmpty() || stack.pop() != pairs.get(c)) {
                return false;
            }
        }
    }
    return stack.isEmpty();
}

```

► Задача 11. Бинарный поиск (Sber #55)

```

public static int binarySearch(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    while (left <= right) {
        // ВАЖНО: защита от overflow
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}

```

Сложность $O(\log n)$. Обязательно использовать $\text{left} + (\text{right} - \text{left}) / 2$ вместо $(\text{left} + \text{right}) / 2$ — иначе при больших значениях будет overflow.

► Задача 12. Подсчёт символов в строке

```
public static Map<Character, Integer> countChars(String s) {
    Map<Character, Integer> counts = new HashMap<>();
    for (char c : s.toCharArray()) {
        counts.merge(c, 1, Integer::sum);
    }
    return counts;
}

// Альтернатива через Stream
public static Map<Character, Long> countCharsStream(String s) {
    return s.chars()
        .mapToObj(c -> (char) c)
        .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
}
```

► Задача 13. Реверс связного списка

«Дан head односвязного списка. Верни head перевёрнутого». Часто просят и итеративно, и рекурсивно.

Итеративно

```
class Node {
    int val;
    Node next;
    Node(int val) { this.val = val; }
}

public static Node reverse(Node head) {
    Node prev = null;
    Node curr = head;
    while (curr != null) {
        Node nextTmp = curr.next; // запомнили следующий
        curr.next = prev;        // развернули
        prev = curr;             // продвинулись
        curr = nextTmp;
    }
    return prev;
}
```

Рекурсивно

```
public static Node reverseRecursive(Node head) {
    if (head == null || head.next == null) return head;
    Node newHead = reverseRecursive(head.next);
    head.next.next = head;
    head.next = null;
    return newHead;
}
```

► Задача 14. Сортировка пузырьком

Часто просят написать сортировку и обсудить сложность. Пузырёк — самая простая.

```
public static void bubbleSort(int[] arr) {
```

```

int n = arr.length;
boolean swapped;
for (int i = 0; i < n - 1; i++) {
    swapped = false;
    for (int j = 0; j < n - 1 - i; j++) { // после i-го прохода последние i
отсортированы
        if (arr[j] > arr[j + 1]) {
            int tmp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = tmp;
            swapped = true;
        }
    }
    if (!swapped) break; // оптимизация: ранний выход
}
}

```

Сложность: $O(n^2)$ в среднем и худшем случае, $O(n)$ в лучшем (уже отсортированный — выйдем после первого прохода). По памяти — $O(1)$.

► Задача 15. Стек на массиве и связном списке

На массиве

```

public class ArrayStack<T> {
    private Object[] arr;
    private int top = -1;
    private static final int DEFAULT = 16;

    public ArrayStack() { arr = new Object[DEFAULT]; }

    public void push(T value) {
        if (top + 1 >= arr.length) {
            arr = Arrays.copyOf(arr, arr.length * 2);
        }
        arr[++top] = value;
    }

    @SuppressWarnings("unchecked")
    public T pop() {
        if (top == -1) throw new NoSuchElementException("Stack is empty");
        T value = (T) arr[top];
        arr[top--] = null;
        return value;
    }

    public boolean isEmpty() { return top == -1; }
}

```

На связном списке

```

public class LinkedStack<T> {
    private static class Node<T> {
        T value;
        Node<T> next;
        Node(T value, Node<T> next) { this.value = value; this.next = next; }
    }

    private Node<T> head;
}

```

```
public void push(T value) {
    head = new Node<>(value, head);
}

public T pop() {
    if (head == null) throw new NoSuchElementException();
    T value = head.value;
    head = head.next;
    return value;
}

public boolean isEmpty() { return head == null; }
}
```


16 SQL live coding

Сбер любит давать SQL-задачи. Обычно дают 2 таблицы и просят 1–2 запроса. Всё уровня Junior — JOIN, GROUP BY, HAVING. Ниже — типовые задачи, которые регулярно встречаются.

► Задача 1. Базовый JOIN с агрегатом

«Есть таблицы users(id, name) и orders(id, user_id, amount, created_at). Вернуть для каждого пользователя его имя и общую сумму заказов».

```
SELECT u.name, COALESCE(SUM(o.amount), 0) AS total
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
GROUP BY u.id, u.name;
```

LEFT JOIN — чтобы видеть пользователей и без заказов. COALESCE — превращает NULL (когда заказов нет) в 0. GROUP BY и по id, и по name — на случай одинаковых имён, чтобы не схлопнуть строки.

► Задача 2. HAVING — фильтр по агрегату

«Найти пользователей с более чем 5 заказами за последний месяц».

```
SELECT user_id, COUNT(*) AS order_count
FROM orders
WHERE created_at >= NOW() - INTERVAL '1 month'
GROUP BY user_id
HAVING COUNT(*) > 5
ORDER BY order_count DESC;
```

WHERE фильтрует строки ДО группировки, HAVING — после, по агрегатам. Это важная разница: COUNT(*) > 5 не работает в WHERE — там агрегат ещё не вычислен.

► Задача 3. Найти дубли

«Найти все email-адреса, которые встречаются больше одного раза в таблице users».

```
SELECT email, COUNT(*) AS cnt
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

► Задача 4. Вторая по величине зарплата (классика Сбера)

«Найти вторую по величине зарплату из employees».

Простой способ

```
SELECT MAX(salary) AS second_max
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Через DISTINCT + LIMIT/OFFSET

```
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
```

```
LIMIT 1 OFFSET 1;
```

Через оконные функции

```
SELECT salary
FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) t
WHERE rnk = 2
LIMIT 1;
```

DENSE_RANK даёт ту же позицию для одинаковых зарплат — это правильнее. ROW_NUMBER присваивает уникальные номера и при дубликатах вторая зарплата может оказаться первой по факту.

► Задача 5. Найти пользователей без заказов

```
SELECT u.*
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
WHERE o.id IS NULL;
```

Классика анти-JOIN. LEFT JOIN даёт всех пользователей + соответствующие заказы (или NULL, если заказов нет). WHERE o.id IS NULL отсеивает тех, у кого заказы есть, оставляя только «одиночек».

► Задача 6. Топ-3 заказа на каждого пользователя

```
SELECT *
FROM (
    SELECT o.*,
           ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY amount DESC) AS rn
    FROM orders o
) t
WHERE rn <= 3;
```

► Задача 7. Категории с продуктами (Sber Инсайд)

«Таблицы category(id, name) и product(id, category_id, price, name). Серия запросов:»

а) Топ-3 категории по числу продуктов

```
SELECT c.name, COUNT(p.id) AS product_count
FROM category c
LEFT JOIN product p ON p.category_id = c.id
GROUP BY c.id, c.name
ORDER BY product_count DESC
LIMIT 3;
```

б) Категории без продуктов

```
SELECT c.*
FROM category c
LEFT JOIN product p ON p.category_id = c.id
WHERE p.id IS NULL;
```

с) Средняя цена продукта в каждой категории

```
SELECT c.name, AVG(p.price) AS avg_price
FROM category c
JOIN product p ON p.category_id = c.id
```

```
GROUP BY c.id, c.name;
```

17 Code Review (что любят давать в Сбере)

На Сбере любят дать кусок Spring-сервиса и попросить отревьюить. Это не формальность — проверка, что ты узнаёшь типичные проблемы Junior-кода. Ниже — самые частые замечания с примерами.

► Структура устного ревью

1. Прочитай код за минуту, спроси контекст: «что это за сервис?», «как используется?».
2. Дай общее впечатление: «вижу проблемы трёх типов: инъекция, обработка ошибок, ресурсы».
3. Пройдись по строчкам, конкретно.
4. В конце предложи, как бы переписал.

► Топ ошибок Junior-кода

1. @Autowired на поле вместо constructor injection

```
// ❌ Плохо
@Service
public class UserService {
    @Autowired
    private UserRepository repo;
}

// ✅ Хорошо
@Service
public class UserService {
    private final UserRepository repo;

    public UserService(UserRepository repo) {
        this.repo = repo;
    }
}
```

Замечание: constructor injection даёт final-поля (иммутабельность), удобнее тестировать, не позволяет циклические зависимости.

2. System.out.println вместо логгера

```
// ❌ Плохо
System.out.println("User created: " + user.getId());

// ✅ Хорошо
private static final Logger log = LoggerFactory.getLogger(UserService.class);
log.info("User created: {}", user.getId());
```

Замечание: параметризованный лог не делает конкатенацию строк, если уровень отключён. Управляется через logback / log4j2. System.out в проде не настраивается.

3. double для денег

```
// ❌ Плохо
double price = 100.0;
double total = price * 0.13; // потеря точности!
```

```
//  Хорошо
BigDecimal price = new BigDecimal("100.00");
BigDecimal total = price.multiply(new BigDecimal("0.13"))
    .setScale(2, RoundingMode.HALF_EVEN);
```

Замечание: $0.1 + 0.2$ в double даёт 0.30000000000000004. Для денег НИКОГДА не использовать double.

4. Optional.isPresent / get вместо функционального стиля

```
//  Плохо
Optional<User> u = repo.findById(id);
if (u.isPresent()) {
    return u.get().getName();
} else {
    return "Гость";
}

//  Хорошо
return repo.findById(id)
    .map(User::getName)
    .orElse("Гость");
```

5. findAll().stream().filter — вместо запроса в БД

```
//  Плохо: вытаскивает ВСЕ записи и фильтрует в памяти
boolean exists = repository.findAll().stream()
    .anyMatch(u -> u.getEmail().equals(email));

//  Хорошо: один SQL EXISTS, без загрузки в память
boolean exists = repository.existsByEmail(email);
```

6. JDBC Connection через new вместо пула

```
//  Плохо
Connection conn = DriverManager.getConnection(url, user, password);

//  Хорошо: через DataSource с пулом (HikariCP)
@Autowired
private DataSource dataSource;

try (Connection conn = dataSource.getConnection()) { ... }
```

Замечание: пул соединений (HikariCP) переиспользует подключения. Каждое новое подключение через DriverManager — дорого, тысячи в секунду повалят БД.

7. synchronized на методе без многопоточности

```
//  Зачем тут synchronized?
public synchronized List<User> getAll() {
    return repository.findAll();
}
```

Замечание: если метод не работает с shared mutable state — synchronized лишний. Он только замедляет (контроль монитора, барьер памяти) и ничего не защищает.

8. Public-поле вместо приватного с геттером

```
//  Плохо
public class User {
    public String name;
```

```

}

// ✅ Хорошо
public class User {
    private String name;
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}

```

Замечание: инкапсуляция. Если потом захочется добавить валидацию в setName или ленивую инициализацию getName — public-поле этого не даст.

9. catch (Exception) вместо конкретного

```

// ❌ Плохо: ловим всё подряд, прячем баги
try {
    process();
} catch (Exception e) {
    log.error("Error", e);
}

// ✅ Хорошо: ловим конкретное
try {
    process();
} catch (IOException e) {
    log.error("IO error", e);
    throw new ServiceException("Не удалось прочитать", e);
}

```

Замечание: catch (Exception) ловит даже RuntimeException, которые сигнализируют о багах в коде. Их надо чинить, а не глотать.

10. Mutable shared state без синхронизации

```

// ❌ Опасно в Spring (бин синглтон!)
@Service
public class CounterService {
    private int counter = 0;
    public void increment() { counter++; } // race condition
    public int get() { return counter; }
}

// ✅ Через Atomic
@Service
public class CounterService {
    private final AtomicInteger counter = new AtomicInteger(0);
    public void increment() { counter.incrementAndGet(); }
    public int get() { return counter.get(); }
}

```

► Code Review в Сбере — процесс

- Сколько аппрувов нужно на merge в Сбере?
 - Обычно 2 аппрува от backend-разработчиков + финальный просмотр начальника отдела перед влитием. Это compliance-требование (важная сумма банковских регуляторов).

ФИШКА · Что говорить на вопрос «как ты делаешь ревью»

Подготовь ответ: «(1) Сначала проверяю корректность — делает ли код то, что в задаче. (2) Потом — обработку ошибок и null. (3) Потом — соответствие стилю проекта и SOLID. (4) Если

есть БД — нет ли N+1. (5) Если многопоточно — нет ли race condition. (6) В последнюю очередь — мелочи именования. И комментарии оставляю конкретные с предложением, а не «переделай».

На стажёре ревью реально не делают, но вопрос «как бы ты делал» — частый. Готовый ответ даёт +1 в карму.

18 Пет-проекты для портфолио

Один работающий пет-проект на GitHub стоит десяти строк в анкете. Сбер сам говорит: руководители смотрят на курсовые, хакатоны, самописные программы. Один-два проекта в портфолио резко повышают шанс попасть на интервью. Ниже — идеи под уровень стажёра.

ФИШКА · Объём — не главное

Не нужен enterprise-проект на 50 микросервисов. 1500–3000 строк кода, понятный README, хотя бы базовые тесты, всё на GitHub — этого достаточно.

Лучше один доделанный пет, чем десять заброшенных.

► Идея 1. REST API для библиотеки книг

Классика для стажёра. Простое CRUD-приложение, на котором можно показать весь базовый стек.

Что внутри

- REST API: POST /books (создать), GET /books (список с пагинацией), GET /books/{id}, PUT /books/{id}, DELETE /books/{id}.
- Spring Boot 3 + Spring Web + Spring Data JPA.
- PostgreSQL в Docker (docker-compose).
- Liquibase для миграций.
- DTO для запросов/ответов, маппинг через MapStruct или вручную.
- Валидация через @Valid + Hibernate Validator (@NotBlank, @Size).
- Глобальная обработка ошибок через @ControllerAdvice.
- Юнит-тесты сервисов с Mockito, интеграционные с MockMvc + @DataJpaTest.
- README с командами для запуска и примерами curl.

Что покажешь на собеседе

- Что знаешь Spring Boot, JPA, REST.
- Что умеешь думать о валидации и обработке ошибок.
- Что писал тесты — это сразу плюс на собеседе.

► Идея 2. Сокращалка URL (типа bit.ly)

Чуть интереснее, чем CRUD. Можно показать понимание алгоритмов и кэширования.

- POST /shorten — принимает URL, возвращает короткий код.
- GET /{code} — 302 редирект на оригинальный URL.
- Генерация коротких кодов: base62 от ID или хэш (объяснить выбор).
- Postgres для хранения.
- Опционально: Redis-кэш для горячих кодов (показывает понимание кэширования).
- Метрика: счётчик переходов по каждому коду.

► Идея 3. To-do list с пользователями

- Регистрация / логин через Spring Security + JWT.

- Каждый пользователь видит только свои задачи.
- CRUD для задач + теги + дедлайны.
- Postgres + Liquibase.
- Тесты на авторизацию (не должен видеть чужие задачи).

Хорош для демонстрации Spring Security базово (что часто спрашивают, хотя в гайде не разбирали).

► **Идея 4. Парсер RSS-лент / агрегатор новостей**

- Шедулер (@Scheduled) опрашивает RSS-ленты раз в N минут.
- Сохраняет новости в БД, дедуплицирует по URL.
- REST API: GET /news?source=...&since=... с пагинацией.
- Showcase: работа с внешними HTTP-сервисами (RestTemplate / WebClient), шедулирование, дедупликация.

► **Идея 5. Учебный «банк»**

Релевантно Сберу — можно прямо упомянуть на собеседовании как мотивацию.

- Сущности: User, Account, Transaction.
- Операции: создать счёт, пополнить, перевести между счетами, история операций.
- BigDecimal для денег (отдельный плюс на собеседовании банка!).
- Транзакции (@Transactional) для атомарности перевода: списать с одного и зачислить на другой — либо оба, либо ничего.
- Защита от двойного списания: если упало посередине, баланс не должен испортиться.

► **Что НЕ делать**

ЛОВУШКА · Типичные ошибки в pet-проектах

- (1) Заброшенный after 3 коммитов — выглядит как «начал и бросил».
- (2) README на одну строку «My Spring Boot app» — не показывает, что внутри.
- (3) Нет тестов — на собеседовании спросят, скажешь «не успел», минус.
- (4) 50% кода — generated boilerplate из IntelliJ-шаблона.
- (5) «Запускается только в IntelliJ» — нужен хотя бы docker-compose up.

► **Структура хорошего README**

- Что это и какую проблему решает — 1 абзац.
- стек: Java 21 / Spring Boot 3 / Postgres / Liquibase / JUnit 5.
- Архитектурная схема — даже если простая, нарисованная от руки в Excalidraw.
- Как запустить: docker-compose up + ./mvnw spring-boot:run.
- Примеры запросов: curl-команды с примерами JSON.
- Тесты: ./mvnw test.
- Что бы доделал дальше — показывает рефлексию.

ФИШКА · О pet-проекте на собеседовании

Подготовь рассказ на 2–3 минуты: что делает, почему именно это, какие были сложности, как решал.

Хорошие фразы: «Здесь я столкнулся с N+1 и решил через @EntityGraph», «Долго думал, как защитить переводы от race condition».

Это сразу показывает, что не просто скопировал tutorial, а реально разобрался.

19 Soft skills и рассказ о себе (STAR)

На стажировке soft skills важны на одном уровне с техникой. Сбер сам подчёркивает: способность учиться, командность, способность думать вслух. И первое впечатление — рассказ о себе — формирует фон для всего интервью.

► Рассказ о себе

- Расскажи о себе — что говорить?

→ Готовь рассказ на 2–3 минуты. Структура: (1) Кто ты — имя, курс, вуз, специальность. (2) Как пришёл в Java — через что попал в программирование, какие курсы/книги читал. (3) Один-два конкретных проекта или достижения (учебный проект, хакатон, олимпиада, своё приложение). (4) Почему интересно именно Java backend и почему Сбер. (5) Чего хочешь от стажировки.

ФИШКА · Не нужно перечислять всё резюме

Самое частое — кандидат за 5 минут пересказывает свою анкету. Это плохо: интервьюер уже её прочитал.

Лучше выбрать 2–3 ключевых момента и подать их ярко. «Я делал X, в процессе столкнулся с Y, решил так-то».

► STAR — для рассказов о проектах

- Что такое STAR?

→ Методика структурированного рассказа о ситуации. S — Situation (контекст: что за проект, кто в команде). T — Task (твоя задача в этой ситуации). A — Action (что конкретно ты делал). R — Result (что получилось, какие выводы). Сбер прямо рекомендует эту технику в материалах для стажёров.

Пример рассказа по STAR

S (Ситуация): На 3 курсе делали командный проект — приложение для организации турниров по настолкам. Команда 4 человека, 2 недели.

T (Задача): Мне нужно было сделать backend на Spring Boot — REST API для управления турнирами и игроками, и наладить деплой.

A (Действия): Сделал API на Spring + Postgres, накатил миграции через Liquibase. Столкнулся с проблемой одновременной записи (двое подтверждали матч — итог затирался). Решил через @Version (оптимистичная блокировка). Деплой настроил через docker-compose.

R (Результат): Проект сдали на отлично, попал в топ-3 на defrag. Понял, как работают транзакции БД и почему важна атомарность.

► Топовые soft-вопросы

- Какой пет-проект делал? Что было самым интересным?

→ Ответ через STAR. Главное — не просто описать, что сделал, а показать рефлексию: что было сложно, чему научился, что бы переделал.

- Почему именно Сбер?

→ Готовы 2–3 факта: масштаб (один из крупнейших IT-работодателей России), Platform V (своя облачная платформа), SberAI (GigaChat, Kandinsky), 78% оффер после стажировки. Можешь упомянуть конкретный продукт Сбера, которым пользуешься. НЕ говорить «потому что зарплата хорошая».

- Что знаешь о Сбере как о работодателе?

→ Экосистема: СберМаркет, Самокат, Okko, СберЗдоровье, СберДевайсы, Сбер Бизнес. Platform V — собственная low-code платформа. SberAI с GigaChat и Kandinsky. Программа стажировок Sberseasons. Активный набор студентов: 6000+ стажёров ежегодно.

- Готов ли совмещать стажировку с учёбой?

→ Честно. Если готов — скажи, что обычно посвящаешь учёбе X часов в неделю, стажировке готов отдать Y. Если есть гибкость в расписании — упомяни. Сбер понимает, что стажёр — студент.

- Где видишь себя через год?

→ Развиваться как Java-разработчик: вырасти от стажёра до junior'a / middle, освоить более глубокие темы (микросервисы, многопоточность, БД). Если есть конкретное направление — упомяни. Главное — показать рост, а не «не знаю, как пойдёт».

- Что делаешь, если задача непонятна?

→ Алгоритм: (1) Перечитываю требования, формулирую вопрос. (2) Гуглю / читаю документацию. (3) Спрашиваю коллегу или ментора. (4) Не пропадаю на полдня молча — приношу промежуточный результат и обсуждаю. Главное — не блокировать команду и не делать «непонятно но как-то».

- Что если конфликт с коллегой?

→ Подход: (1) Сначала пробую решить 1-на-1, спокойно. (2) Стараюсь понять его сторону — может, я не так понял. (3) Если не получается — иду к тимлиду, не за «он плохой», а за «помогите разрулить процесс». Конструктивный, неэмоциональный подход. Не нужно говорить «у меня никогда не было конфликтов» — это не вызывает доверия.

- Опыт в Scrum / Agile?

→ Если в учебном проекте делали daily-стендапы и спринты — это уже опыт. Можно сказать: «В курсовой работали по двухнедельным спринтам, делали ретро в конце». Если не делали — честно: «Знаю теорию (роли Scrum Master, Product Owner, спринты, артефакты), на практике не работал, но готов вписаться».

- Готов ли работать с legacy-кодом?

→ Да, и стоит это раскрыть. В банке много legacy — никуда не деться. Хорошие фразы: «Понимаю, что legacy — это часть работы», «Это полезный опыт — понять, как развивается большая система», «Скорее всего, придётся читать чужой код больше, чем писать свой».

► Что читать (если спросят)

- Joshua Bloch «Effective Java» — must-have для Java-разработчика.
- Cay Horstmann «Core Java» — фундаментальный учебник, есть на русском.
- Герберт Шилдт «Java. Полное руководство» — классика на русском.
- Robert Martin «Чистый код» — про code style и качество.
- «Грокаем алгоритмы» (Адитья Бхаргава) — самый дружелюбный учебник по алгоритмам.
- LeetCode Easy — для тренировки live coding.
- Telegram-канал JavaJub — реальные вопросы с собесед.

ВНИМАНИЕ · Не говори, что читаешь только Хабр

Хабр — норм, но один он не показывает системного подхода к обучению. Хороший ответ: «По фундаменту читаю Шилдта/Хорстманна, по практике решаю задачи на LeetCode, ленты в Хабре читаю для актуальных новостей».

► Маркеры, которые ценит Сбер

По словам самих интервьюеров Сбера (Хабр):

- Коммуникабельность — задаёшь уточняющие вопросы, не молчишь.
- Готовность рассуждать вслух при решении задач.
- Открытость — не пытаешься скрыть пробелы. Честное «не знаю» — это плюс.
- Пунктуальность — пришёл вовремя на встречу.
- Энтузиазм — видно, что интересно учиться.

И маркеры, по которым ставят минус:

- Плохо отзывается о прежних начальниках или коллегах.
- Не задаёт встречных вопросов — «нет вопросов» это почти отказ.
- Выдумывает ответы вместо честного «не знаю».
- Пользуется AI-помощниками во время собеседования — Сбер прямо просит этого не делать.
- Не пунктуален или ведёт себя пренебрежительно с рекрутером.

► Встречные вопросы — обязательно подготовь

- Над какими проектами работает команда?
- Какой стек используется в команде?
- Как устроен процесс ревью кода и деплоя?
- Есть ли наставник для стажёра? Как часто встречи?
- Какие задачи дают стажёру в первый месяц?
- Что нужно сделать, чтобы получить оффер после стажировки?

20 План подготовки и финальный чек-лист

Если до собеседа 2 недели — этого достаточно. Если меньше — приоритезируй разделы 03 (Java Core), 04 (Коллекции), 07 (SQL) и 15 (практика). Это базовая часть, по которой проверяют в первую очередь.

► Неделя 1 — фундамент

- День 1–2. Java Core: ООП, примитивы, String, equals/hashCode, исключения, Optional, лямбды, Stream API.
- День 3. Коллекции: ArrayList vs LinkedList, устройство HashMap. Сложности операций.
- День 4. JVM: где живут примитивы и объекты, GC, JVM/JRE/JDK.
- День 5. Многопоточность базово: поток vs процесс, synchronized, volatile, deadlock, race condition.
- День 6. SQL: SELECT, JOIN, GROUP BY, HAVING, ACID, уровни изоляции.
- День 7. Прорешать 10 live coding задач из раздела 15. Палиндром, FizzBuzz, развернуть строку, дубли.

► Неделя 2 — стек и практика

- День 8–9. Spring и Spring Boot: DI, бины, аннотации, @Transactional, автоконфигурация. Hibernate базово: N+1, Lazy/Eager.
- День 10. HTTP / REST: методы, идемпотентность, статус-коды, REST vs SOAP.
- День 11. Git, Maven, паттерны (Singleton, Builder, Factory, Observer, Proxy vs Decorator).
- День 12. Алгоритмы: Big-O, бинарный поиск, сортировки, BFS/DFS, two pointers.
- День 13. Прорешать 10 SQL-задач из раздела 16. Code Review — пройти все 10 ловушек из раздела 17.
- День 14. Pet-проект: довести до состояния, в котором не стыдно показать. README, тесты, docker-compose.

► За день до собеседа

1. Перечитать разделы 03 (Java Core), 04 (Коллекции), 07 (SQL) — самое спрашиваемое.
2. Прорешать ещё 2–3 простых задачи из раздела 15 — чтобы рука была размята.
3. Подготовить рассказ о себе на 2–3 минуты по STAR.
4. Подготовить 3–5 встречных вопросов (раздел 19).
5. Зарядить ноутбук, проверить SberJazz / Zoom, протестировать камеру и микрофон.
6. Лечь спать вовремя.

► В день собеседа

- Не паникуй. Сбес не отбраковывает, а отбирает. Стажёр — это младший уровень, требования к нему адекватные.
- Думай вслух. Молчаливое верное решение хуже разговорчивого неверного.
- Уточняй задачи. «На пустом массиве что возвращать?» — это плюс, а не минус.
- Не знаешь — скажи. «Не знаю точно, но логически предположил бы...» — лучше выдумки.
- Задавай встречные вопросы. Молчание на «есть вопросы?» — почти гарантированный отказ.

- Не пользуйся AI и не открывай поисковики. Сбер прямо просит этого не делать.

► **Финальный чек-лист**

Блок	Готов, если можешь...
Java Core	объяснить контракт equals/hashCode, разницу final/finally/finalize, что такое JVM/JRE/JDK, как работает Optional и Stream API.
Коллекции	рассказать устройство HashMap (бакеты, treeify), почему ArrayList быстрее LinkedList, как обойти ConcurrentModificationException.
JVM	сказать, где живут примитивы и объекты, что делает GC, разницу StackOverflowError и OutOfMemoryError.
Многопоточность	знать, как создать поток, что такое synchronized, volatile, deadlock, race condition. Объяснить, почему i++ не атомапен.
SQL	написать SELECT с JOIN, GROUP BY и HAVING. Знать ACID, уровни изоляции. Объяснить, что такое индекс и почему нельзя на всё.
Spring	объяснить IoC/DI, что такое @Autowired, скоупы бинов, как работает @Transactional и его подвох с self-invocation.
Hibernate	знать JPA vs Hibernate, основные аннотации, что такое Lazy/Eager, N+1, LazyInitializationException и как их решать.
HTTP/REST	перечислить HTTP-методы и их идемпотентность, разницу 401/403, что такое REST и чем отличается от SOAP.
Git и Maven	знать базовые команды Git, что такое Pull Request, жизненный цикл Maven.
Паттерны	рассказать про Singleton, Builder, Factory, Observer. Объяснить разницу Proxy и Decorator.
Тесты	написать простой тест с JUnit 5 + Mockito. Объяснить разницу Mock и Spy.
Алгоритмы	перечислить сложности коллекций, написать бинарный поиск с защитой от overflow, объяснить BFS vs DFS.
Live coding	уверенно решать 10 базовых задач: палиндром, FizzBuzz, развернуть строку, найти дубли, скобки, Two Sum.
SQL практика	написать запрос со JOIN + GROUP BY + HAVING, найти пользователей без заказов, найти вторую зарплату.
Code Review	узнавать 10 типовых ошибок: field injection, double для денег, findAll вместо findByX, catch Exception.
Pet-проект	показать один рабочий проект на GitHub с README, тестами, docker-compose.
Soft skills	рассказать о себе за 2–3 минуты по STAR, иметь 3–5 встречных вопросов, объяснить почему Сбер.

Удачи на собеседе!

// git push origin offer

JavaJub